分 类 号_____ 学号_____
学校代码_____ 密级_____

華中科技大學

硕士学位论文

(学术型□ 专业型☑)

基于驱动程序替换的物联网设备固件仿真技术研究

学位申请人：

学 科 专 业：

指 导 教 师：

答 辩 日 期：

A Dissertation Submitted in Partial Fulfillment of the Requirements for

Research on IoT Device Firmware Emulation Technology Based on Driver Replacement

Candidate :

Major :

Supervisor :

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

March, 2026

独创性声明

本人声明所呈交的学位论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除文中已标明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权华中科技大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本论文属于 保　密 □，在 ____ 年解密后适用本授权书。
不保密 □。

（请在以上方框内打“√”）

学位论文作者签名：　　指导教师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

随着物联网技术的快速发展，物联网设备数量呈现爆炸式增长，其固件安全问题日益突出。固件仿真技术是进行固件安全测试和漏洞挖掘的重要手段，然而现有的固件仿真方法存在外设建模成本高、硬件依赖强、自动化程度不足等问题。传统的基于硬件建模的方法需要为每种外设手动建立仿真模型，工作量大且难以覆盖多样化的硬件生态；基于驱动层替换的方法虽然避免了底层硬件建模，但驱动函数的定位与替换仍高度依赖人工干预，难以实现规模化应用。

针对上述问题，本文提出一种结合大语言模型的自动化固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型的代码理解与生成能力自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的稳定运行。

本文的主要工作包括三个方面。首先，设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块，该模块通过 MMIO 模式识别、数据流追踪与递归归纳技术，从固件中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。其次，构建了基于大语言模型的驱动函数分析与替换模块，该模块利用大语言模型理解驱动函数的语义，通过结构化提示词模板和函数分类策略，自动生成可在仿真环境中运行的替代代码，并通过编译修复循环保障代码的可编译性。最后，引入了动态测试反馈闭环机制，当固件在仿真环境中运行出现异常时，系统能够自动分析异常原因并反馈给驱动替换模块，通过”生成—测试—反馈—修正”的闭环流程持续优化仿真质量。

实验结果表明，本文方法能够有效支持多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）的固件仿真，显著降低了固件仿真的技术门槛和人工成本，为后续的模糊测试和漏洞挖掘奠定了基础。

关键词：物联网安全；固件仿真；驱动程序替换；大语言模型；静态分析

Abstract

With the rapid development of Internet of Things (IoT) technology, the number of IoT devices has grown explosively, and firmware security issues have become increasingly prominent. Firmware emulation is an important approach for firmware security testing and vulnerability discovery. However, existing firmware emulation methods face challenges such as high peripheral modeling costs, strong hardware dependencies, and insufficient automation. Traditional hardware modeling-based methods require manually building simulation models for each peripheral, which is labor-intensive and difficult to cover the diverse hardware ecosystem. Although driver layer replacement-based methods avoid low-level hardware modeling, the identification and replacement of driver functions still heavily rely on manual intervention, making it difficult to achieve large-scale application.

To address these issues, this paper proposes an automated firmware emulation method combined with large language models (LLMs). The core idea is to automatically identify hardware-dependent code in firmware through static analysis, use the code understanding and generation capabilities of LLMs to automatically generate alternative driver implementations, and build a dynamic testing feedback loop to iteratively optimize emulation quality, thereby enabling firmware to run stably in emulation environments without manually building peripheral models.

The main contributions of this paper include three aspects. First, a hardware dependency code automatic identification module for embedded firmware is designed and implemented, which automatically extracts hardware-related function call points, peripheral register access patterns, and driver function boundaries from firmware through MMIO pattern recognition, data flow tracking, and recursive induction techniques, constructing a queryable driver knowledge base. Second, a driver function analysis and replacement module based on LLMs is built, which utilizes LLMs to understand driver function semantics, automatically

generates alternative code that can run in emulation environments through structured prompt templates and function classification strategies, and ensures code compilability through a compilation repair loop. Finally, a dynamic testing feedback loop mechanism is introduced. When firmware runs abnormally in the emulation environment, the system can automatically analyze the cause of the abnormality and feed it back to the driver replacement module, continuously optimizing emulation quality through a ”generate-test-feedback-revise” closed-loop process.

Experimental results show that the proposed method can effectively support firmware emulation for multiple embedded platforms (STM32, NXP) and operating systems (FreeRTOS, RT-Thread, bare metal), significantly reducing the technical barrier and manual cost of firmware emulation, laying the foundation for subsequent fuzzing and vulnerability discovery.

Keywords: IoT Security; Firmware Emulation; Driver Replacement; Large Language Model; Static Analysis

目　　录

摘　要 I

Abstract II

1　绪论

1.1　研究背景与意义 ..... (1)

1.2　国内外研究现状 ..... (3)

1.3　研究内容与主要贡献 ..... (9)

1.4　论文组织结构 ..... (10)

2　背景知识与相关技术

2.1　物联网设备硬件基础 ..... (13)

2.2　物联网操作系统与驱动 ..... (14)

2.3　大模型代码分析智能体框架 ..... (15)

2.4　静态分析技术 ..... (17)

2.5　固件仿真与模糊测试 ..... (18)

2.6　本章小结 ..... (19)

3　系统设计与实现

3.1　驱动函数接口静态分析与识别模块 ..... (21)

3.2　驱动函数分析、替换与编译模块 ..... (29)

3.3　动态测试反馈闭环模块 ..... (40)

3.4　本章小结 ..... (46)

4　实验设计与结果分析

4.1　实验设计 ..... (48)

4.2　功能测试 ..... (49)

4.3　性能测试 ..... (58)

4.4　本章小结 ..... (63)

5 总结与展望

5.1 工作总结 ..... (64)

5.2 论文创新点 ..... (65)

5.3 论文贡献 ..... (66)

5.4 局限性与不足 ..... (66)

5.5 未来工作展望 ..... (67)

致 谢

攻读硕士学位期间发表的学术论文 ..... (70)

1 绪论

1.1 研究背景与意义

近年来，随着物联网（Internet of Things, IoT）技术迅速发展，物联网设备已被广泛用于智能家居、智慧城市、新能源、车联网、医疗健康等多个应用领域。根据GSMA 发布的《The mobile economy 2020》报告显示，2019 年全球物联网总连接数达到 120 亿，预计到 2025 年，全球物联网总连接数规模将达到 246 亿，年复合增长率高达 13%[?]。物联网操作系统是一种面向物联网设备和应用的轻量级嵌入式操作系统，它可以提供更好的设备管理、数据处理和安全性能。目前，市场上有许多物联网操作系统，例如华为的 OpenHarmony[?]、亚马逊的 FreeRTOS[?]、Linux 基金会的 Zephyr[?] 等。然而，物联网和嵌入式操作系统面临着严峻的安全挑战[?]，如恶意攻击、数据泄露、隐私侵犯等，这些安全问题可能会给用户带来损失和危害。在真实世界中发生的物联网安全问题也层出不穷。2018 年，一款名为”Mirai” 的恶意软件[?] 利用了 IoT 设备的弱点，通过攻击嵌入式设备的操作系统，将其变成僵尸网络的一部分，对互联网上的服务器发起了大规模的 DDoS 攻击。俄乌冲突期间，乌克兰遭遇了俄罗斯发起的破坏性网络攻击，该攻击利用一种名为 Industroyer 的恶意软件控制电力系统中的开关和断路器，意图瘫痪乌克兰的电力系统。这些安全事件的不断发生表明，当前物联网和嵌入式操作系统安全问题的研究刻不容缓。

固件仿真（Firmware Re-Hosting）技术是一种克服固件分析硬件依赖问题的有效方法，它通过在商用计算机和服务器的软件模拟器上运行嵌入式系统固件，实现对嵌入式设备的软件级模拟，使得包含嵌入式系统的固件无需实际硬件即可进行安全测试[?]。然而，当前固件仿真技术的**自动化程度仍然严重不足**，这主要体现在以下几个方面：首先，现有的外设仿真方法大多需要人工编写模型或基于有限规则的自动推断。硬件在环方案需要为每种外设手动配置物理设备连接；基于原始硬件的外设建模方案需要人工分析数据手册并编写仿真代码；基于程序分析的方案虽然能自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。其次，

驱动层替换方案（如 HALucinator）虽然利用了微控制器制造商提供的硬件抽象层（HAL），通过函数 Hook 的方式进行替换，但驱动函数的定位、语义理解和替换代码生成仍然需要大量人工干预。分析人员需要手动识别固件中哪些部分存在硬件依赖并需要替换，理解驱动函数与硬件的交互逻辑，并编写相应的替代代码，这个过程不仅耗时耗力，而且容易出错。这种**自动化程度不足导致的人工成本高昂**，严重制约了固件仿真技术在大规模安全测试中的应用，成为当前物联网设备安全领域亟待解决的关键问题。

现有固件仿真技术的自动化不足主要体现在三个核心环节。在驱动函数定位环节，识别固件中哪些部分存在硬件依赖并需要替换是一项具有挑战性的任务。在嵌入式程序中，硬件依赖代码往往分散在各个模块中，通过函数调用、全局变量访问、寄存器操作等多种形式存在，手动定位这些代码需要分析人员对固件结构和硬件特性有深入理解，过程既耗时又容易出错。在外设建模环节，当前的自动化方法只能处理简单的外设行为，对于网络控制器、USB 设备等具有多通道、DMA 传输、中断嵌套等特性的复杂外设，往往需要人工编写仿真模型。在替换代码生成环节，现有工具主要通过函数 Hook 返回预设值或执行简化逻辑，缺乏对驱动函数内部语义的深入理解，无法准确模拟硬件行为，需要人工编写复杂的替代代码。这三个环节的自动化不足相互叠加，使得固件仿真过程对分析人员经验的依赖程度过高，难以适应大规模、多样化的固件安全测试需求。

针对上述问题，本研究致力于提升固件仿真技术的自动化程度，降低人工成本，实现高效的物联网设备固件安全测试。具体而言，本研究提出一种基于驱动程序替换的固件仿真方法，通过静态分析技术自动识别固件中的硬件依赖代码，利用大语言模型的代码理解能力深入分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。该方法的核心创新在于将自动化技术贯穿于驱动函数定位、语义分析和代码生成的全过程，显著减少了对分析人员人工干预的依赖。通过提升固件仿真的自动化水平，本研究不仅能够降低固件安全测试的成本，还能够提高测试效率和覆盖率，为物联网设备的大规模安全测试提供技术支撑，具有重要的理论意义和应用价值。

1.2 国内外研究现状

固件是嵌入式设备中的软件组成部分，通常包含了设备的操作系统、驱动程序、应用程序和配置文件等。固件的安全性对于保护设备和用户数据的完整性和隐私至关重要。然而，由于固件的复杂性、多样性和闭源性，对固件进行有效的安全测试是一项具有挑战性的任务。

基于固件仿真的模糊测试技术在近年来受到了越来越多的关注和研究，出现了一些代表性的工具和系统，如 FIRMADYNE[?]、FUZZWARE[?] 等。这些工具和系统在不同程度上实现了对固件仿真和模糊测试的支持，并在实际的固件中发现了一些新的漏洞。然而，基于固件仿真的模糊测试技术仍然存在一些挑战和问题，如仿真环境的准确性、测试用例的有效性等，需要进一步的改进和优化。下面将从固件仿真技术、嵌入式设备模糊测试技术两个方面阐述当前基于固件仿真的嵌入式设备固件模糊测试技术研究现状。

1.2.1 固件仿真技术研究现状

当前固件仿真的主要挑战是为模拟器中没有实现的硬件提供有效的输入。诸如 QEMU[?] 之类的仿真器提供了模拟各种 CPU 体系结构的能力，但对商用嵌入式系统中使用的各种各样的外设，例如计时器、UART、以太网控制器等，提供的支持却十分有限，而这些外设对于正确实现固件仿真却是十分必要的。

固件仿真技术的发展经历了从依赖真实硬件到全自动化模拟的演进过程。早期研究采用硬件在环（Hardware in the Loop, HIL）的方法，如 Zaddich 等人提出的 Avatar[?] 和 Muench 等人提出的 Avatar2[?]，通过混合执行的方式将固件分析过程中对外设的访问转发到物理硬件中。然而，这种方案仍然依赖实际设备，且仿真环境和物理设备之间的切换影响了运行速度。为摆脱对真实硬件的依赖，研究者提出了多种全自动化仿真方案，主要包括以下三类。

1.2.1.1 基于原始硬件的外设建模方案

解决外设依赖的一种方法是在底层内存映射寄存器接口上实现硬件，或通过观察真实硬件的行为来构建外设模型。然而这是一项费力的任务，扩展性不好。当前

主流的仿真器 QEMU 也仅能仿真 10 余种设备，远不能满足固件仿真工作的需求。

Gustafson 等人提出的 Pretender[?] 工具通过观察固件与物理设备之间真实发生的交互记录，使用机器学习、模式识别等方法为每个外设构建模型，再结合 QEMU 或 angr 等仿真引擎实现固件仿真。这种方法需要真实设备的支持，但在观测到足够的交互模式后，可以自动生成较为准确的外设模型。

1.2.1.2 基于程序分析的外设建模方案

基于程序分析的外设建模方案通过对固件代码进行静态或动态分析，提取固件与外设之间的交互逻辑，构建虚拟外设模型以替代真实硬件。

Feng 等人提出的 P2IM[?] 工具通过动态符号执行和模糊测试，自动识别固件中的输入点和输出点，构建与真实设备相似的虚拟模型。Cao 等人提出的 Laelaps[?] 在因访问未实现的外设而卡住时，切换到符号执行以找到正确输入，引导 QEMU 找到与实际执行最相近的路径。Zhou 等人提出的 uEmu[?] 工具利用符号执行分析固件镜像，将未知的外设寄存器表示为符号，构建推断规则并存储在知识库中。Scharnowski 等人提出的 Fuzzware[?] 使用精确的 MMIO 建模实现有效的固件模糊测试，通过确定固件逻辑如何实际使用硬件生成的值，自动生成模型以提高模糊测试输入数据的有效性。

1.2.1.3 基于系统或驱动层替换的外设仿真方案

基于系统或驱动层替换的仿真技术，又称高级仿真（High Level Emulation, HLE），利用固件中的公共抽象来消除为外设提供低级支持的需要。Chen 等人提出了一个针对 Linux 内核的物联网设备固件仿真平台 Firmadyne[?]，通过修改内核和驱动来添加硬件支持，实现物联网设备的全系统仿真。该工具解决了定制化外围 I/O、非易失性内存、动态文件等仿真技术难题。Li 等人创建了 Para-rehosting[?] 工具，该工具的主要思想是利用 POSIX 接口抽象并实现了一个便携式 MCU，以将嵌入式系统源代码重新编译到商用机器的本机可执行文件中，以便利用 x86 上许多现有的复杂软件测试工具对代码进行测试。然而，这种方案的实现需要分析师获得固件样本的源代

码，而在第三方的固件分析工作中，源代码的获取往往是困难甚至完全不可行的。Clements 等人提出了 HALucinator[?] 工具，该工具利用了微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过对硬件抽象层函数的抽象与替换进行固件仿真，这项工作在重托管简单的裸金属固件程序时取得了很好的效果。

然而，上述基于系统或驱动层替换的仿真技术自动化程度较差，难以适应大规模安全测试的需要。具体而言，这类技术面临以下两个核心难点：一是难以自动化定位需要进行替换的驱动程序。在嵌入式程序中，识别哪些部分存在硬件依赖并需要替换，往往依赖分析人员的手动干预。手动分析嵌入式程序的复杂性使得这一过程既耗时又容易出错。二是难以自动化生成替换程序。除了识别硬件相关功能，当前的技术也缺乏自动化生成替代程序的能力。分析人员必须手动分析函数的功能，理解其与硬件和中间件的交互，并编写相应的替代代码。这个过程不仅繁琐，而且容易导致人力成本的增加。

1.2.1.4 固件仿真技术对比

表 1.1 对比了当前主流固件仿真技术的优缺点。

表 1.1 当前主流固件仿真技术对比

方案	典型工具	优点	缺点
基于原始硬件的外设建模	QEMU, Pretender	可对外设行为进行精确模拟，提高测试效率和覆盖率	需要为每种外设专门建模，工作量大，通用性差，依赖真实设备观测
基于程序分析的外设建模	P2IM, uEmu, Laelaps, Fuzzware	可根据程序自身信息自动推断外设行为，减少人工干预，提高通用性	难以完全覆盖复杂外设的所有行为，可能存在误报或漏报
基于系统或驱动层替换	HALucinator, Firmadyne, Para-rehosting	在系统层或驱动层对函数进行抽象和替换，降低对硬件的依赖，提高测试速度和灵活性	驱动函数的定位与替换依赖人工干预，自动化程度不足，难以适应大规模安全测试

在这三类方案中，基于系统或驱动层替换的固件仿真方法是一种有前景的方案。

它从固件系统层和驱动函数入手，较好地解决了基于程序分析和原始硬件建模的方法对复杂外设难以完全模拟或建模工作量大等痛点。HALucinator 等工具利用微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过函数替换的方式实现固件仿真，在重托管简单的裸金属固件程序时取得了很好的效果。

然而，现有驱动层替换方案在自动化方面仍存在明显不足：一方面，驱动函数的定位依赖分析人员的手动干预，识别固件中哪些部分存在硬件依赖并需要替换是一个耗时且容易出错的过程；另一方面，替换程序的生成也缺乏自动化支持，分析人员需要手动分析函数功能、理解其与硬件的交互，并编写相应的替代代码。为解决上述问题，需要设计一种能够自动化定位需要替换的驱动函数、并利用大模型或代码分析技术自动生成替代程序的方案，以提升固件仿真过程的自动化程度，实现高效的物联网设备程序仿真与动态测试。

1.2.2 嵌入式系统模糊测试技术发展现状

嵌入式系统模糊测试技术是一种针对嵌入式设备的软件安全测试方法，它可以自动或半自动地生成随机或变异的输入数据，发送给嵌入式设备的 Web 服务或其他接口，监控设备的运行状态，发现潜在的漏洞或缺陷。目前，嵌入式系统模糊测试技术主要以下两个方向：面向真实设备的黑盒模糊测试和基于固件仿真的灰盒模糊测试。

1.2.2.1 面向真实设备的黑盒模糊测试

直接对嵌入式设备进行模糊测试，不需要对设备进行任何修改或插桩，只需要获取设备的网络接口或串口等信息，然后使用模糊测试工具生成并发送测试用例，监控设备的响应和异常。Chen 等人开发的 IoTFuzzer[?] 工具，就利用了应用程序与物联网设备的交互信息，通过网络接口向嵌入式设备发送模糊测试数据，进行模糊测试工作。这种方法的优点是简单易用，不依赖于设备的内部结构或代码，适用于黑盒测试场景。缺点是测试效率较低，覆盖率较差，难以发现深层次的漏洞。

1.2.2.2 基于固件仿真的灰盒模糊测试

基于固件仿真的模糊测试，通过对嵌入式设备的固件进行逆向工程，提取出可执行的二进制代码，然后使用仿真器在虚拟环境中运行固件，并对其进行模糊测试。基于该方案的仿真工具有 Fuzzware[?]、uEmu[?]、HALucinator[?] 等。这种方法的优点是可以对固件进行插桩或修改，获取更多的运行信息，提高测试效率和覆盖率，发现更多的漏洞。缺点是需要对固件进行复杂的逆向分析，可能遇到加密或保护机制，同时仿真器可能存在不完全或不准确的问题，导致测试结果与真实设备不一致。

1.2.3 大模型程序分析与代码生成技术研究现状

大语言模型（Large Language Model, LLM）在代码理解和生成方面展现出了强大的能力。近年来，研究者开始探索将 LLM 应用于程序分析和代码生成领域，涌现出多种增强策略以提升模型在复杂编程任务中的表现。

1.2.3.1 面向特定任务的代码生成方法

早期 LLM 通过零样本提示能够生成基础代码，但在处理复杂编程任务时准确性和可靠性仍存在不足。为克服这些局限，研究者发展了以下增强策略：

指令微调（Instruction Tuning）：通过对预训练 LLM 在特定领域数据集上进行精细调整，显著提升模型对特定编程任务的理解和执行能力[?]。DolphCoder 模型通过多样化的指令和自评估机制，结合代码评估目标有效提升了代码生成能力[?]。CodeLSI 框架则通过低秩优化与领域特定指令微调相结合，提高了代码生成的领域特异性和成本效益[?]。

测试驱动的迭代生成：利用测试用例作为反馈信号指导模型逐步修正生成的代码，直至通过所有测试。这种方法通过隐式执行监督信号，结合搜索、重排序或迭代修正等机制动态优化模型输出[?]。RLCF（Reinforcement Learning with Compiler Feedback）方法通过编译器反馈对 LLM 进行额外训练，解决生成代码中可能存在的语言级不变量违规问题[?]。CodeDPO 框架则将偏好学习整合到代码生成过程中，优化模型优先选择正确且高效的解决方案[?]。

多步推理方法：思维链（Chain-of-Thought, CoT）和程序思维（Program-of-Thought,

PoT）等方法通过将复杂问题分解为多个可管理的子步骤，提升模型解决复杂问题的能力[?]。MoTCoder 引入模块化思维指令框架来应对更具挑战性的编程任务[?]。程序思维方法则将中间推理步骤编译为可执行代码片段，实现了”推理即执行”，大幅提升了数值计算和 API 调用类任务的准确性。

工具增强的大模型：通过集成编译器、调试器、静态分析工具等外部工具，扩展模型的实际应用能力。这种范式将 LLM 定位为”AI 协调者”，通过标准化工具调用协议调度各种工具，形成感知-决策-执行的闭环[?]。MCCoder 系统专注于生成运动控制代码，并结合严格的验证机制以确保代码安全性[?]。RGD 代理调试器利用多 LLM 通过细化和生成指导来自动修复代码中的错误[?]。

1.2.3.2 系统级软件代码生成的挑战与前沿

在系统级软件应用层面，特别是针对物联网设备驱动程序、操作系统内核和嵌入式固件等复杂系统，LLM 的代码生成面临独特挑战。IoT 设备驱动程序开发具有强硬件耦合性、并发中断敏感性、底层资源约束性以及隐式的 DMA/寄存器交互等特征，对 LLM 的代码生成能力提出了更高要求[?]。

当前前沿研究强调构建”分析—生成—验证”的闭环系统以应对这些挑战：

静态分析集成：静态分析工具如 CIL、Smatch 和 Frama-C 用于提取接口契约与控制流约束，对代码进行结构化表示[?]。这些工具能够识别不安全的功能、缓冲区溢出和竞态条件等安全漏洞，为 LLM 生成提供结构化上下文。

动态符号执行验证：动态符号执行引擎如 QEMU 和 KLEE 能够生成高覆盖率的路径约束，并反向验证生成代码的内存安全与中断语义正确性[?]。通过结合编译器诊断、安全扫描和符号执行，LLM 可以迭代细化其输出。

端到端流水线：研究者正在构建端到端的”分析 → 生成 → 符号验证 → 反馈精炼”流水线[?]。形式化规约被用作生成约束，而验证失败轨迹则被结构化为自然语言反馈以驱动下一轮生成。Agents4PLC 框架通过基于 LLM 的代理自动化了可编程逻辑控制器（PLC）代码生成与验证，提高了工业控制系统的操作效率和安全性。

这些研究为本文利用 LLM 进行驱动函数分析和替换提供了理论基础，特别是在

构建自动化分析-生成-验证闭环方面具有重要的参考价值。

1.2.4 现有研究的局限性分析

综合上述研究现状，现有固件仿真技术主要存在以下局限性：

（1）外设建模依赖人工，难以处理复杂外设行为

现有仿真方法大多依赖人工编写外设模型或基于有限规则的自动推断。硬件在环方案需要为每种外设手动配置物理设备连接；基于原始硬件的外设建模方案需要人工分析数据手册并编写仿真代码；基于程序分析的方案虽然能自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。例如，网络控制器、USB 设备等复杂外设具有多通道、DMA 传输、中断嵌套等特性，现有自动推断方法往往只能处理简单的 MMIO 寄存器读写，无法准确模拟这些复杂交互行为。

（2）缺乏对驱动函数语义的深入理解，仿真结果可能不准确

现有驱动层替换方案（如 HALucinator）主要通过函数 Hook 的方式进行替换，即当固件调用某个 HAL 函数时，返回预设的模拟值或执行简化的模拟逻辑。然而，这种方法缺乏对驱动函数内部语义的深入理解：函数之间的调用关系、参数的数据流传递、外设状态的变化等关键信息往往被忽略。这导致在某些场景下，仿真结果与真实硬件行为存在偏差，可能遗漏重要的程序路径或引入虚假的漏洞。

（3）缺乏自动化反馈机制，难以迭代优化仿真质量

当前大多数仿真工具采用”一次性配置”的模式：分析人员手动配置仿真环境后运行测试，如果发现问题则需要人工介入调试和修复。这种模式缺乏自动化的反馈闭环机制——当仿真过程中出现异常（如程序卡死、断言失败、外设访问超时）时，系统无法自动分析原因并调整仿真策略。这导致仿真质量的提升高度依赖分析人员的经验，难以适应大规模、多样化的固件测试需求。

1.3 研究内容与主要贡献

为解决上述问题，本文提出一种基于驱动程序替换的物联网设备固件仿真方法。该方法通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。

本文主要的创新点和贡献有三个：

（1）本研究设计并实现了一个面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别、数据流追踪等技术，从固件二进制代码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。与现有依赖人工标注或有限规则推断的方法相比，该模块能够自动处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，显著降低了驱动函数定位的人工成本。

（2）本研究设计并实现了一个基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，本模块能够处理不同类型的驱动函数（如初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。与现有基于预设返回值的简单 Hook 方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

（3）本研究引入了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种” 生成-测试-反馈-修正” 的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

1.4 论文组织结构

本文共分为五章，从绪论、背景知识、系统设计与实现、实验验证到总结展望，逐步深入地阐述基于驱动程序替换的物联网设备固件仿真方法。图 1.1 展示了本文的整体组织结构。

第一章：绪论。阐述物联网设备安全问题的研究背景，分析固件仿真技术在物联网安全测试中的作用。梳理国内外在固件仿真、嵌入式系统模糊测试、大模型程

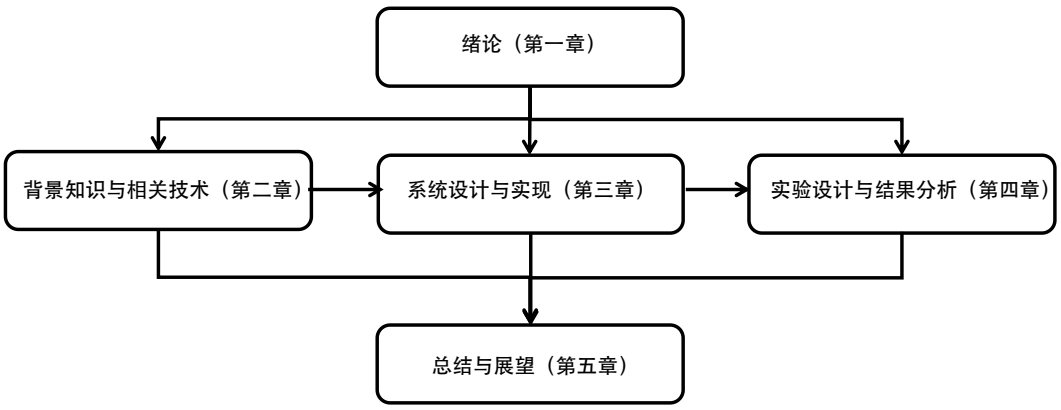

图 1.1 论文组织结构图

序分析与代码生成等领域的研究现状，分析现有技术存在的局限性，明确本文的研究内容和主要贡献。

第二章：背景知识与相关技术。介绍物联网设备硬件基础（ARM Cortex-M 架构、MMIO）、物联网操作系统与驱动（RTOS、HAL）、大模型代码分析智能体框架（工作流范式、LLM 框架）、静态分析技术（CodeQL）以及固件仿真与模糊测试技术（QEMU、AFL），为后续章节奠定理论基础。

第三章：系统设计与实现。详细阐述基于驱动程序替换的固件仿真方法的设计与实现。首先介绍系统的整体架构设计，包括静态分析模块、大语言模型驱动分析模块、代码生成模块、动态测试反馈模块等核心组件的职责划分和数据流转关系，然后详细描述各关键模块的实现细节。

第四章：实验设计与结果分析。通过实验验证本文方法的有效性和实用性。首先介绍实验环境配置和实验设计，包括评价指标、对比实验和消融实验，然后展示实验结果，从多个维度分析系统性能，最后对实验结果进行深入讨论。

第五章：总结与展望。总结全文工作，概括本文的主要创新点和贡献，回顾基于驱动程序替换的固件仿真方法的研究成果。客观分析当前工作中存在的不足和局

限性，展望未来的研究方向，为后续研究提供参考。

2 背景知识与相关技术

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动框架、大模型代码分析技术以及固件仿真与模糊测试技术，为后续章节的论述奠定基础。

2.1 物联网设备硬件基础

物联网设备通常采用嵌入式微控制器（Microcontroller Unit, MCU）作为核心处理器。常见的 MCU 架构包括 ARM Cortex-M 系列、RISC-V 等。这些处理器具有低功耗、低成本、实时性强等特点，广泛应用于物联网领域。本节重点介绍 ARM Cortex-M 架构的特性以及内存映射 I/O 机制。

2.1.1 ARM Cortex-M 架构特性

ARM Cortex-M 是 ARM 架构处理器核心中低端系列的统称，专为低成本、高能效的微控制器设计，广泛应用于物联网设备。该系列采用 Thumb-2 指令集，在提供 32 位性能的同时保持 16 位代码密度。Cortex-M 架构提供了多项增强实时性和可靠性的特性，包括嵌套向量中断控制器（NVIC）用于中断管理、系统定时器用于提供可编程的周期性中断并支持操作系统的任务调度、以及主栈与进程栈的栈模式切换机制以实现多任务并发和内存保护。

2.1.2 内存映射 I/O

内存映射 I/O（Memory-Mapped I/O, MMIO）是嵌入式系统中外设与处理器通信的主要方式。MMIO 将外设寄存器映射到处理器的统一内存地址空间中，处理器通过普通的内存读写指令（如 LDR/STR）访问特定内存地址，即可完成对外设的控制和数据传输。这种设计简化了编程模型，使得外设访问与内存访问具有相同的语法形式。

ARM Cortex-M 架构采用统一的 32 位地址空间，按功能划分为代码区（Flash）、SRAM 区、外设区和系统区。外设区是 MMIO 的核心区域，所有片上外设的寄存器

都被映射到此区域，不同厂商的外设被分配到不同的地址段。系统区位于地址空间高端，包含 NVIC、SysTick、MPU 等系统级组件的控制寄存器。理解 MMIO 地址空间布局对于固件仿真至关重要，仿真器需要针对外设区和系统区实现专门的访问处理逻辑。

外设寄存器按照功能可分为控制寄存器、状态寄存器和数据寄存器三类。控制寄存器用于配置外设的工作模式和参数，如 USART 的 CR1 可设置波特率、数据位长度等。状态寄存器反映外设的当前工作状态，如 USART 的 SR 包含发送缓冲区空（TXE）、接收缓冲区非空（RXNE）等标志位，其值由硬件自动更新。数据寄存器用于数据传输，如 USART 的 DR 用于发送和接收数据，访问数据寄存器通常会触发外设执行相应操作。

2.2 物联网操作系统与驱动

2.2.1 实时操作系统概述

实时操作系统（Real-Time Operating System, RTOS）是物联网设备中广泛采用的管理任务调度和资源分配的软件系统。RTOS 能够及时响应外部事件或数据，并在规定的时间内完成任务，其工作原理是通过一个实时任务调度器来管理系统资源，按照优先级或者时间片来分配 CPU 时间给不同的任务，保证重要或者紧急的任务优先执行。

物联网领域常见的 RTOS 包括 FreeRTOS[?]、Zephyr[?]、RT-Thread[?] 等。FreeRTOS 是 Amazon 开源的实时操作系统，专为微控制器和小型微处理器设计，内核精简，最小可运行在不到 10KB 的 RAM 上，是物联网领域使用最广泛的 RTOS 之一。Zephyr 是 Linux 基金会管理的开源实时操作系统，支持多种处理器架构，提供更完整的操作系统功能，包括文件系统、网络协议栈、设备驱动模型等，采用模块化设计以平衡功能丰富性和资源占用。RT-Thread 是中国开源社区主导开发的实时操作系统，采用微内核架构，并提供了丰富的组件生态。

2.2.2 硬件抽象层

硬件抽象层（Hardware Abstraction Layer, HAL）是嵌入式软件架构中的关键组成部分，它位于硬件驱动层和上层应用之间，为应用程序提供统一的硬件访问接口。HAL 层的核心目标是屏蔽底层硬件的差异性，使得上层应用代码可以在不同的硬件平台之间移植，而无需修改业务逻辑。

在传统的嵌入式开发中，应用程序需要直接操作寄存器来完成外设控制，这种方式不仅代码可读性差，而且当更换硬件平台时需要大量修改。HAL 层通过定义统一的接口规范，将硬件操作封装为函数调用，显著提高了代码的可维护性和可移植性。例如，不同厂商的 UART 外设寄存器布局可能完全不同，但通过 HAL 层，应用程序可以使用统一的 `HAL_UART_Transmit()` 函数来发送数据，而无需关心底层寄存器的具体实现。

HAL 层的接口设计遵循统一的命名和功能规范。以 STM32 HAL 库为例，每个外设类型提供初始化函数（如 `HAL_UART_Init`）、数据传输函数（如 `HAL_UART_Transmit`）、状态查询函数（如 `HAL_UART_GetState`）和中断管理函数（如 `HAL_UART_Receive_IT`）。由于 HAL 函数封装了底层硬件操作细节，仿真器可通过替换 HAL 层函数来实现硬件行为模拟，无需深入理解寄存器语义，相比寄存器级模拟更加高效且易于实现。

2.3 大模型代码分析智能体框架

大语言模型（Large Language Model, LLM）在代码理解和生成方面展现出强大的能力，为自动化程序分析提供了新的技术途径。本节重点阐述代码分析工作流范式和基于 LLM 的智能体开发框架，为后续章节的驱动函数分析和替换工作奠定技术背景。

2.3.1 代码分析工作流设计模式

基于 LLM 智能体框架构建代码分析系统时，研究者提出了多种工作流设计模式。这些模式为处理代码分析任务中常见的逻辑推理、工具调用、多步骤协作等挑战提供了系统化思路。

ReAct 模式：ReAct（Reasoning + Acting）将推理与行动交替进行[?]。在代码分析中，LLM 可先推理分析目标（如" 需要理解该函数的调用关系"），再调用静态分析工具获取信息，观察结果后继续下一步推理。该模式使 LLM 能够动态收集信息并逐步完成复杂分析。

规划-执行模式：该模式将任务分为规划与执行两阶段。规划阶段生成步骤化的执行计划，执行阶段按序完成。其优势在于结构清晰、便于调试和过程监控。

多智能体协作模式：该模式使用多个专门化的智能体协同完成任务[?]。例如在驱动函数替换中，可设计分析智能体、生成智能体、验证智能体与协调智能体，通过消息传递或共享状态完成复杂代码分析与生成任务。本研究在驱动函数替换模块中，综合运用了 ReAct 模式与多智能体协作模式，以应对硬件相关代码替换的复杂性与正确性要求。

2.3.2 LLM 智能体开发框架

为了更好地利用大语言模型构建复杂的应用系统，研究人员开发了多种智能体（Agent）框架。这些框架提供了结构化的方式来组织 LLM 的调用、管理状态、集成外部工具，使开发者能够构建可靠的、可扩展的 LLM 应用。

2.3.2.1 LangChain 框架

LangChain 是目前最流行的 LLM 应用开发框架之一，提供了模型、提示模板、链、代理、工具和记忆等核心组件。模型组件封装了与不同 LLM 提供商的交互接口，便于切换和比较。提示模板允许开发者定义带变量的提示词，在运行时动态填充，提高复用性。链将多个组件按顺序组合，构建复杂的处理流程。代理是更高级的抽象，允许 LLM 根据用户输入自主选择合适的工具（如搜索引擎、代码执行器）并决定后续行动。工具是代理可以调用的外部功能，框架提供了大量内置工具，开发者也可自定义。记忆组件维持应用在多次调用之间的状态，例如在多轮对话中存储历史对话。

2.3.2.2 LangGraph 状态图

LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库。与 LangChain 的链式结构不同，LangGraph 基于状态图的概念，状态在各个节点之间传递和更新。节点是处理单元，边定义转换关系并支持条件分支和循环，适用于迭代优化等场景。LangGraph 还支持检查点机制，可将工作流状态持久化存储，支持长时间运行任务的暂停和恢复。

2.4 静态分析技术

静态分析技术是软件工程和安全研究的重要基础，旨在通过在不执行程序的情况下，通过分析源代码或二进制代码来推断程序的属性。静态分析的优势在于能够覆盖程序的所有可能执行路径，不受测试用例的限制。常见的静态分析技术包括数据流分析、控制流分析、指针分析、抽象解释等。

2.4.1 CodeQL 代码分析平台

CodeQL 是 GitHub 开发的语义级代码分析平台，支持多种编程语言（包括 C/C++、Java、Python、JavaScript 等）的安全漏洞检测和代码质量分析。CodeQL 的核心思想是将源代码转换为可查询的数据库，然后通过声明式查询语言（QL）进行灵活的分析。与传统基于规则的静态分析工具不同，CodeQL 提供了表达力丰富的查询语言，研究人员可以通过编写查询来查找特定的代码模式、追踪数据流、分析控制依赖等复杂问题。

CodeQL 的工作原理分为两个阶段。第一阶段是数据库构建，CodeQL 对源代码进行解析，生成抽象语法树（AST）、控制流图（CFG）、数据流图（DFG）等多层次的结构化表示，并将这些信息存储在数据库中。第二阶段是查询执行，研究人员使用 QL 语言编写查询，在数据库上执行，获取分析结果。CodeQL 的查询语言基于 Datalog，采用谓词逻辑的表达方式，能够自然地表达”所有存在某种模式的代码片段”或”从源点到汇点的数据流路径”等复杂查询。

CodeQL 在程序分析领域应用广泛，包括安全漏洞检测（如 SQL 注入、缓冲区溢出）、代码质量分析（如死代码检测、复杂度分析）、以及依赖关系分析（如函数

调用图、数据流追踪）。本研究利用 CodeQL 的数据流追踪能力，建立 MMIO 访问与函数参数之间的关联关系，为驱动函数识别和语义分析提供支撑。

2.5 固件仿真与模糊测试

固件仿真是物联网安全研究的重要技术手段。由于物联网设备的硬件多样性和封闭性，直接在真实设备上进行安全测试面临诸多困难。固件仿真技术通过软件模拟硬件环境，使研究人员能够在通用计算平台上运行和分析嵌入式固件，从而降低测试成本并提高效率。模糊测试作为一种高效的漏洞挖掘技术，与固件仿真相结合，能够在仿真环境中自动化地发现固件中的安全漏洞。

2.5.1 QEMU 仿真器

QEMU[?] 是目前应用最广泛的开源机器仿真器，支持多种处理器架构和外设仿真，其设计目标是提供高性能、高可移植性的系统仿真能力，使其成为固件仿真研究的重要基础平台。QEMU 支持两种主要的仿真模式：用户模式仿真（User Mode Emulation）允许在一种架构上运行为另一种架构编译的单个程序，适用于分析用户空间程序；系统模式仿真（System Mode Emulation）提供完整的硬件系统仿真，包括处理器、内存、中断控制器、外设等组件，可以运行完整的操作系统或裸机固件，这是固件仿真的主要工作模式。

QEMU 的核心是动态二进制翻译引擎（Tiny Code Generator, TCG），它将目标架构的指令块翻译为中间表示（IR），然后再将 IR 翻译为宿主机指令，这种两级翻译机制使 QEMU 能够支持多种目标架构和宿主架构的组合。为了提高性能，TCG 采用即时编译（JIT）技术，将翻译结果缓存在内存中供后续执行重复使用。

2.5.2 模糊测试技术

模糊测试（Fuzzing）是一种自动化软件测试技术，通过向目标程序输入大量随机、半随机或结构化的测试用例，来发现程序中的安全漏洞和缺陷。模糊测试的基本工作流程包括：生成或变异测试用例、将测试用例输入目标程序、监控程序执行状态（如崩溃、断言失败、内存错误等）、记录和分析异常。

American Fuzzy Lop（AFL）[?] 是最具影响力的覆盖率引导模糊测试工具之一。

AFL 的核心思想是通过代码插桩技术记录程序执行过程中的覆盖率信息，利用这些信息指导测试用例的变异和选择，优先保留能够触发新执行路径的测试用例，从而在有限的测试时间内探索更多的程序状态，提高漏洞发现的概率。AFL 的工作流程包含两个核心环节：覆盖率记录和变异策略。在覆盖率记录环节，AFL 通过编译时插桩或运行时插桩（利用 QEMU 的动态翻译功能）在目标程序中插入覆盖率记录代码，记录程序执行过程中经过的基本块和边。在变异策略环节，AFL 维护一个测试用例队列，不断从队列中选择测试用例，对其进行变异，执行变异后的测试用例，并根据覆盖率变化决定是否将新用例加入队列。AFL 的变异策略包括位翻转、字节删除、字节插入、块拼接、算术运算等多种操作，能够产生多样化的测试输入。通过这种迭代式的测试和筛选机制，AFL 能够逐步探索程序的各种执行路径，自动发现潜在的漏洞和缺陷。

在嵌入式固件模糊测试领域，AFL 常与仿真器（如 QEMU）结合使用。研究者在 QEMU 的动态翻译机制基础上实现插桩，使 AFL 能够对在仿真环境中运行的固件进行模糊测试。这种方法不需要修改固件源码，能够直接对二进制固件进行测试，显著提高了测试的自动化程度和可扩展性。

2.6 本章小结

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动、大模型代码分析智能体框架、静态分析技术、固件仿真与模糊测试技术，为后续章节的论述奠定基础。

3 系统设计与实现

本章详细阐述基于驱动程序替换的固件仿真系统的设计与实现。系统采用”静态识别 → 智能替换 → 构建验证 → 动态反馈”的分层架构，通过 CodeQL 静态分析和大语言模型智能体技术，实现固件驱动函数的自动识别、语义分析和代码替换，最终在仿真环境中运行并支持模糊测试。

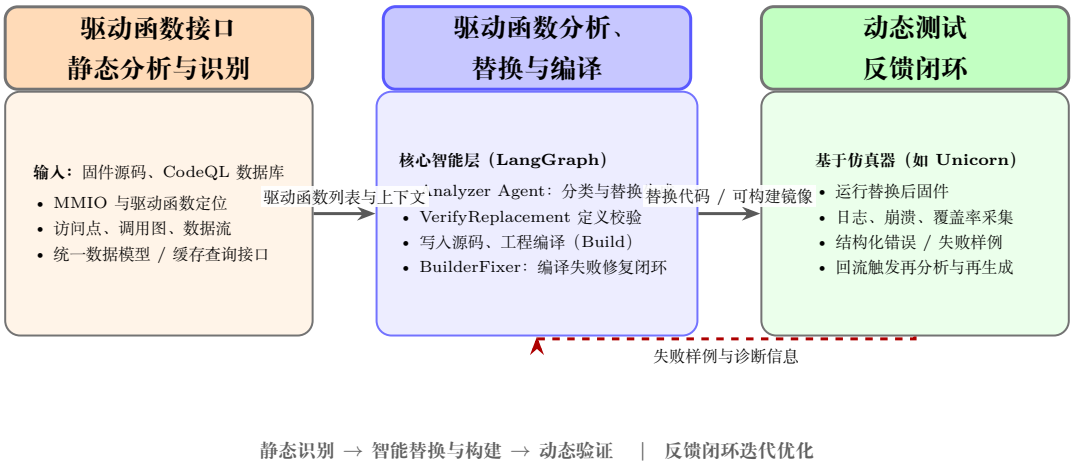

图 3.1 系统整体架构图

由图 3.1可见，系统在整体上划分为三个核心部分：左侧为驱动函数接口静态分析与识别模块，中间为驱动函数分析、替换与编译模块，右侧为动态测试反馈闭环模块。以下先概括各模块的职责与数据交接关系，随后从第一节起依次展开实现细节。

左侧的驱动函数接口静态分析与识别模块负责对固件源码进行静态分析，定位硬件相关代码，抽取驱动函数、MMIO 访问点、依赖数据结构、调用关系等信息，并以统一的数据模型对外提供查询接口。该模块基于 CodeQL 静态分析引擎实现，通过预定义的查询规则从固件源码中提取驱动相关信息，其输出结果作为中间模块的输入。

中间的驱动函数分析、替换与编译模块是系统的核心智能层。该模块接收静态分析模块输出的驱动函数列表，使用大语言模型对驱动函数进行类型分类与语义

保留分析，生成可替代实现，同时将替代实现集成回工程并驱动构建。该模块采用 LangGraph 智能体框架实现复杂的推理和决策流程，确保生成的替换代码既满足语义保持性要求，又能够在仿真环境中正确运行。

右侧的动态测试反馈闭环模块负责在仿真环境中运行替换后的固件，采集运行日志、崩溃信息与覆盖率等指标，并将失败样例与错误信息结构化回流，触发重新分析与再生成。该模块是系统质量保障的关键环节，通过构建”生成-测试-反馈-修正”的闭环机制，使仿真质量能够持续提升，减少对分析人员经验的依赖。

三个模块之间通过标准化的数据接口进行交互，形成完整的处理流水线。静态分析模块向替换模块提供驱动函数列表和相关信息，替换模块生成替换代码后传递给测试模块进行验证，测试模块的反馈结果又会触发替换模块进行修正，形成迭代优化循环。

3.1 驱动函数接口静态分析与识别模块

针对嵌入式固件中驱动函数与硬件交互位置难以自动识别的问题，本研究采用静态分析技术构建驱动函数接口静态分析与识别模块。该模块从固件源码中提取硬件依赖信息，构建固件的”硬件依赖画像”，为后续的驱动函数替换提供准确的基础数据。

模块的核心职责是建立从源码到硬件依赖信息的映射，识别固件中与硬件交互的函数，确定函数访问硬件的方式（如直接寄存器访问、HAL 库调用或间接句柄操作），并分析函数之间的调用依赖关系。模块采用静态分析技术构建包含语法树、控制流图和数据流图的综合数据库，通过精心设计的查询规则识别 MMIO 结构体实例和字段访问，进而对涉及硬件访问的函数进行语义分类。数据流追踪技术通过污点追踪建立寄存器访问与函数参数之间的关联，为替换模块提供精确的语句定位信息。模块输出的”硬件依赖画像”包含驱动函数列表、MMIO 访问点、函数调用关系图和数据流信息，这些结构化的分析结果被存储在统一的数据模型中，为下游模块提供可查询的知识库。

3.1.1 MMIO 访问模式识别

MMIO（Memory-Mapped I/O，内存映射 I/O）是一种将硬件外设的寄存器映射到处理器内存地址空间的机制。通过 MMIO，软件可以通过访问内存地址的方式与硬件外设进行交互，而不需要特殊的 I/O 指令。嵌入式厂商通常使用 C 语言结构体来定义 MMIO 寄存器映射，结构体的每个字段对应一个硬件寄存器，字段的偏移量与硬件规范中的寄存器地址偏移保持一致。

MMIO 结构体的典型代码示例如下：

```
// STM32 UART寄存器映射示例
typedef struct {
    __IO uint32_t CR1;     // 控制寄存器1
    __IO uint32_t CR2;     // 控制寄存器2
    __IO uint32_t CR3;     // 控制寄存器3
    __IO uint32_t BRR;     // 波特率寄存器
    __IO uint32_t GTPR;    // 保护时间与预分频寄存器
    __IO uint32_t RTOR;    // 接收超时寄存器
    __IO uint32_t RQR;     // 请求寄存器
    __IO uint32_t ISR;     // 中断与状态寄存器
    __IO uint32_t ICR;     // 中断标志清除寄存器
    __IO uint32_t RDR;     // 接收数据寄存器
    __IO uint32_t TDR;     // 发送数据寄存器
} USART_TypeDef;

// 通过强制类型转换创建MMIO实例
#define USART1_BASE    (0x40011000UL)
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;
```

Listing 3.1 MMIO 结构体示例

定位 MMIO 结构体实例是识别硬件相关代码的关键步骤。本研究总结出三种典型的 MMIO 实例定位模式：

模式一：直接地址强转。这类代码将固定外设基地址强制转换为外设寄存器结构体指针，常见于寄存器定义的头文件中。在代码中呈现为类似 `#define USART1 ((USART_TypeDef *)USART1_BASE)` 的宏定义，后续代码直接对该指针进行字段访问如 `USART1->DR`、`USART1->CR1`。

```
// 驱动源码示例：直接地址强转模式
#define USART1_BASE 0x40011000UL
#define USART1 ((USART_TypeDef *)USART1_BASE)
#define USART2 ((USART_TypeDef *)USART2_BASE)
#define USART3 ((USART_TypeDef *)USART3_BASE)

void uart_init_direct(void) {
    USART1->CR1 = 0x2000;  // 直接对强转实例进行寄存器访问
    USART1->BRR = 0x0341;
}
```

Listing 3.2 直接地址强转模式示例

模式二：间接初始化。这类代码通过句柄（handle）、配置表（descriptor/config table）等间接结构，将外设基地址赋值给某个”寄存器块指针成员”，后续通过该成员完成寄存器访问。在 STM32 HAL 生态中，这类成员常命名为 `Instance`。在代码中呈现为初始化时的赋值语句如 `huart->Instance = COM_USART[COM]`，后续寄存器访问呈现为多级解引用如 `huart->Instance->DR`。在基于配置表的驱动中，也可能呈现为 `handle.Instance = spi_config[i].Instance` 这种从配置表复制的形式。

```c
// 驱动源码示例：间接初始化模式（HAL句柄）
UART_HandleTypeDef huart;

void BSP_COM_Init(COM_TypeDef COM) {
    huart.Instance = COM_USART[COM];  // 将外设基地址赋值给Instance成员
    HAL_UART_Init(&huart);
    // 后续访问呈现为多级解引用
    huart.Instance->DR = data;
}

// 驱动源码示例：间接初始化模式（配置表）
typedef struct {
    USART_TypeDef *Instance;
    uint32_t BaudRate;
} SPI_Config;

SPI_Config spi_config[] = {
    {SPI1, 115200},
    {SPI2, 9600}
};

void rt_hw_spi_bus_init(void) {
    for (int i = 0; i < sizeof(spi_config)/sizeof(spi_config[0]); i++) {
        spi_bus_obj[i].handle.Instance = spi_config[i].Instance;
        // 通过配置表中的Instance赋值
    }
}
```

Listing 3.3 间接初始化模式示例（HAL 句柄和配置表）

模式三：常量指针参数。这类代码通过函数参数接收 MMIO 结构体指针（寄存器块基址），常见于底层驱动或通用外设操作函数。在代码中呈现为函数定义的形参类型是某个寄存器结构体指针，函数体内部通过该参数访问寄存器字段，如 `void mmio_uart_init(USART_TypeDef *uart) { uart->CR1 = 0x2000; }`。调用者可以将模式一或模式二中的 MMIO 实例作为实参传入，如直接调用 `mmio_uart_init(USART1)` 或传入句柄成员 `mmio_uart_init(huart->Instance)`。

```c
// 驱动源码示例：常量指针参数模式
void mmio_uart_init_pointer(USART_TypeDef *uart) {
    // 通过指针访问MMIO寄存器
    uart->CR1 = 0x2000;
    uart->BRR = 0x0341;
}
```

```

// 调用示例
void main(void) {
    // 直接传模式一的强转实例
    mmio_uart_init_pointer(USART1);

    // 传模式二的句柄成员
    mmio_uart_init_pointer(huart.Instance);

    return 0;
}
```

Listing 3.4 常量指针参数模式示例

三种定位模式的对比如表 3.1所示。

表 3.1 MMIO 结构体实例定位模式对比

模式	描述	代码呈现
直接地址强转	将固定地址强制转换为外设结构体指针	整数常量 → 结构体指针类型转换
间接初始化	通过配置表将外设基地址赋值给句柄	结构体成员赋值为外设宏
常量指针参数	函数以常量指针形式接收外设基址	参数类型为 MMIO 结构体指针

3.1.2 驱动函数定位

静态分析模块的核心任务是识别固件中的驱动函数，为后续的智能替换提供目标列表。本研究采用简单直接的定位准则：**凡是包含 MMIO 访问的函数，均定义为驱动函数**。这一定位准则的合理性在于：MMIO 访问是固件与硬件交互的最底层方式，包含寄存器读写的函数必然与硬件强相关，这类函数在仿真环境中无法直接执行，需要进行替换处理；而不包含 MMIO 访问的函数通常是纯软件逻辑，如数据处理、协议解析、状态管理等，在仿真环境中可以直接运行，无需替换。

设 F 为固件中的函数集合，M 为 MMIO 结构体实例集合，$A(f, m)$ 为谓词" 函数 f 访问 MMIO 实例 m 的字段"，则驱动函数集合 D 定义为：

$$D = \{f \in F \mid \exists m \in M : A(f, m)\}$$

驱动函数识别的流程如下：首先通过三种定位模式识别代码中的所有 MMIO 结构体实例；然后对每个 MMIO 结构体实例，追踪其字段访问表达式，记录访问的字段名、访问类型（读/写）、所在语句和所在函数；接着对所有包含 MMIO 字段访问的函数进行汇总，形成驱动函数列表；最后分析驱动函数之间的调用关系，构建函数调用图。

3.1.3 分析与识别模块实现

本节阐述分析与识别模块的具体实现技术。模块采用 CodeQL 静态分析引擎作为底层支撑，CodeQL 通过构建包含语法树、控制流图、数据流图的综合数据库，为语义级代码分析提供了强大的查询能力。

3.1.3.1 MMIO 操作定位模块实现

针对三种 MMIO 定位模式，本研究设计了相应的查询规则。查询从 CodeQL 数据库中提取所有符合 MMIO 结构体特征的变量及其初始化表达式。查询的核心设计思想是将”寄存器字段访问”追溯到一个可被 CodeQL 稳定描述的基对象表达式。

```
import cpp
import libraries.MMIOAnalyzer

from
    Function func,
    InterestingMMIOExpr expr
where
    isInterestingMMIOFunction(func) and
    expr.getEnclosingFunction() = func
select expr.toString(),
       expr.getEnclosingFunction().getName(),
       expr.getFile().getAbsolutePath().toString(),
       expr.getLocation().getStartLine(),
       expr.checkExprType(),
       expr.checkEnclosingType()
```

Listing 3.5 MMIO 表达式收集查询示例

上述查询通过 `import libraries.MMIOAnalyzer` 引入 MMIO 分析库，从 CodeQL 数据库中提取所有有趣的 MMIO 表达式。查询的输出包括表达式字符串、所属函数名、文件路径、行号、表达式类型和封闭类型等信息。

对于”间接初始化”和”常量指针参数”模式，查询采用数据流追踪策略，通过追踪赋值语句和函数参数来识别 MMIO 实例。常量基址的追踪利用 CodeQL 提供的数

据流和污点追踪能力，将”常量大值表达式（外设基址）”追踪到后续的变量访问/解引用位置。

```
predicate isMMIOConstantExpr(ConstantExpr src_mmio) {
    exists( DataFlow::Node source, DataFlow::Node sink, Expr varAccess,
            PointerDereferenceExpr deref |
        source.asExpr() = src_mmio and
        isConstantBigValueExpr(src_mmio) and
        sink.asExpr() = varAccess and
        deref.getOperand() = varAccess and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}

predicate isMMIOTracedExpr(Expr e) {
    exists( DataFlow::Node source, DataFlow::Node sink, ConstantExpr src_mmio |
        source.asExpr() = src_mmio and
        isMMIOConstantExpr(src_mmio) and
        sink.asExpr() = e and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}
```

Listing 3.6 常量基址数据流追踪查询示例

上述查询定义了两个谓词：isMMIOConstantExpr 识别常量大值表达式（外设基址），isMMIOTracedExpr 通过污点追踪将源点追踪到汇点。查询的逻辑可形式化表示为：当存在一个常量大值表达式作为数据流源点，经过数据流传播到达某个变量访问或解引用操作时，该操作即为 MMIO 相关的表达式。

```
// 示例：常量基址在代码中的传播
#define USART1_BASE 0x40011000UL
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;  // 基址强转
huart->Instance = USART1;                               // 赋值给Instance成员
huart->Instance->DR = data;                             // 最终的字段访问点
```

Listing 3.7 常量基址在代码中的传播示例

查询通过污点追踪，将上述代码中的 USART1_BASE（源点）追踪到最终的 huart->Instance->DR（汇点），从而识别出整个 MMIO 使用链路。

对于”函数第 0 个参数是指针类型，实参是常量大值表达式”的情况，查询通过函数调用关系和参数类型匹配，定位”把常量基址传给接受结构体指针的驱动函数”的调用点。

```
predicate isMMIOStructure_MK(Struct s){
    exists( FunctionCall call, Expr e, TypedefType typeDef |
        call.getTarget().getParameter(0).getType() instanceof PointerType and
        (
            (call.getTarget().getParameter(0).getType().(PointerType).getBaseType() =
    typeDef and
             typeDef.getBaseType() = s) or
```

```
            call.getTarget().getParameter(0).getType().(PointerType).getBaseType() = s
        ) and
        call.getArgument(0) = e and
        isConstantBigValueExpr(e)
    )
}
```

Listing 3.8 函数参数类型匹配查询示例

上述查询通过参数类型匹配（PointerType）和实参的常量大值表达式检查，识别出 MMIO 结构体类型的函数调用点。查询的逻辑可表示为：当函数调用的第 0 个参数类型为结构体指针，且该参数对应的实参是经过数据流追踪的 MMIO 常量表达式时，该调用点即为候选的 MMIO 使用点。

```
// 示例：常量指针参数传递
void uart_send_data(USART_TypeDef *uart, uint8_t *data, uint16_t len);

// 调用时传递MMIO实例
uart_send_data(USART1, buffer, len);     // 直接传强转实例
uart_send_data(huart->Instance, buffer, len);  // 传Instance成员
```

Listing 3.9 常量指针参数传递示例

查询通过参数类型匹配（USART_TypeDef *）和实参的数据流追踪，识别出上述调用点中的 MMIO 参数传递关系。

3.1.3.2 驱动函数定位模块实现

驱动函数定位模块的核心任务是根据前述定位准则，从固件源代码中识别出所有驱动函数，并构建驱动函数调用图。模块采用 CodeQL 静态分析技术，通过定义驱动函数的判定规则来实现自动化识别。

模块的核心判定规则是：**包含 DriverFieldAccess 且不包含 MMIOFieldAccess 的函数被定义为驱动函数**。DriverFieldAccess 指的是通过句柄或间接方式访问 MMIO 寄存器块的字段操作，而 MMIOFieldAccess 指的是直接通过 MMIO 结构体指针访问寄存器的字段操作。这一定义确保了识别出的函数是真正需要替换的驱动层函数，而非底层的硬件抽象层函数。

```
class DriverFunction extends Function
{
    DriverFunction() {
        exists(DriverFieldAccess fa | fa.getEnclosingFunction() = this) and
        not exists(MMIOFieldAccess fa | fa.getEnclosingFunction() = this)
    }
}
```

Listing 3.10 驱动函数判定规则

上述查询定义了 DriverFunction 类，该类继承自 Function，通过检查函数内部是否包含 DriverFieldAccess 且不包含 MMIOFieldAccess 来判定函数是否为驱动函数。

在识别出驱动函数列表后，模块进一步分析驱动函数之间的调用关系，构建函数调用图。调用图以驱动函数为节点，函数调用关系为边，记录调用方、被调用方、调用位置等信息。调用图为后续的智能替换模块提供了函数依赖关系，有助于理解函数的调用上下文和语义关联。

```
from DriverToCall call
select call.toString2(),
       call.getTargetFunction().toString(),
       call.getEnclosingFunction().toString(),
       call.isDirectCall()
```

Listing 3.11 驱动函数调用关系查询

上述查询从 DriverToCall 关系中提取驱动函数之间的调用关系，输出调用边的字符串表示、目标函数名、封装函数名以及调用是否为直接调用等信息。

3.1.3.3 统一数据模型

为便于后续模块使用，静态分析结果被组织为统一的数据模型。数据模型采用 Python 数据类（dataclass）定义，支持序列化和反序列化。

Listing 3.12 数据模型定义示例

```
# 示例：数据模型定义
@dataclass
class MmioAccessInfo:
    struct_type: str      # 结构体类型，如"USART_TypeDef"
    field_name: str       # 字段名，如"DR"
    access_type: str      # 访问类型：READ/WRITE
    line_number: int      # 行号
    function_name: str    # 所在函数名
    context_code: str     # 上下文代码

@dataclass
class FunctionInfo:
    name: str             # 函数名
    file_path: str        # 文件路径
    line_number: int      # 行号
    signature: str         # 函数签名
    mmio_accesses: List[MmioAccessInfo]  # MMIO访问列表
```

数据模型包括函数信息模型（包含函数名、文件路径、行号、签名、返回类型、参数列表、分类、源码、调用关系等信息）、MMIO 访问点模型（包含结构体类型、字段名、访问类型、行号、所在函数、上下文代码等）和项目分析结果模型（包含项

目路径、CodeQL 数据库路径、所有函数、MMIO 访问点、函数调用图、驱动函数列表、分析时间戳等）。这些数据模型通过 JSON 序列化存储在缓存层，供后续模块查询和使用。

3.2 驱动函数分析、替换与编译模块

驱动函数分析、替换与编译模块是系统的核心智能层，承担着语义理解和代码生成的核心任务。该模块采用大语言模型驱动的智能体架构，通过 LangGraph 框架构建图式工作流，实现从函数分类、代码生成、质量验证到编译集成的完整自动化流程。该模块采用”语义分析 → 代码生成 → 质量验证 → 编译集成 → 错误修复”的迭代优化模式，通过多轮迭代逐步提升替换质量，直至生成能够在仿真环境中正常运行的替换代码。

3.2.1 分析替换与编译智能体工作流设计

工作流以**单个驱动函数**为粒度调度：每轮从静态分析模块或任务队列中取定一条驱动函数记录（含签名、源码片段、MMIO 访问上下文等），在图式状态机中顺序执行，直至该函数被判定为无需替换并结束，或替换实现通过校验与编译并被采纳。为便于叙述与实现解耦，整条流水线在逻辑上划分为**驱动函数分析与替换代码生成**、**替换代码修正与编译集成**两个子模块；前者由 Analyzer Agent 主导完成上下文拉取、分类与候选替换生成，后者在工程侧完成定义校验、构建与失败修复。整体编排如图 3.2所示。

3.2.1.1 驱动函数分析与替换代码生成模块

该子模块由 **Analyzer Agent** 驱动，在 LangGraph 状态机中通过“工具调用—推理—写回”的循环完成单函数处理。执行顺序固定为：**信息获取** → **代码分类** →（仅在分类结果认定该函数需要替换时）**替换代码生成**。

信息获取阶段通过工具调用拉齐该驱动函数在工程中的可观测上下文，包括完整实现或相关片段、函数签名与参数类型、调用关系与 MMIO 仿真侧提示等（如 GetFunctionInfo、GetFunctionCallStack、GetMMIOFunctionEmulateInfo）。**代码分类**阶

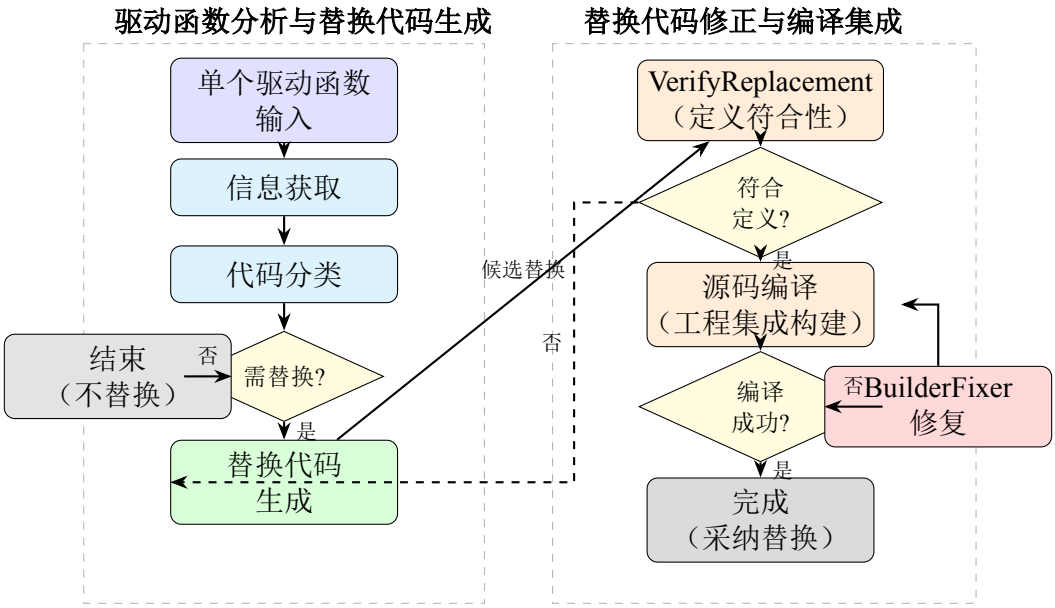

图 3.2 Analyzer Agent 工作流：左侧为分析与替换生成，右侧为定义校验、编译与 BuilderFixer 修复闭环

段综合源码与静态分析结果，由大语言模型判定函数语义类别及是否属于必须替换的硬件相关行为；分类所依据的类别体系及提示构造方式见第 3.2.2 节。**替换代码生成**仅在“需要替换”分支上触发：模型在既定类别上选择替换策略并产出与签名一致的候选实现，具体策略（如 Skip、Strip、Redirect、Event 等）的选择规则与生成约束同样在第 3.2.2 节展开，本节只给出子模块内的先后次序与数据交接关系。

3.2.1.2 替换代码修正与编译集成模块

该子模块在工程目录中对上一子模块输出的**候选替换代码**做**二阶段检验与修复**，目标是在真实构建环境下得到可链接、可编译的替换定义。

第一阶段（定义符合性）调用 VerifyReplacement，依据预置规则与接口契约检查候选替换是否满足替换定义（例如签名一致、禁止引入不兼容的存储类或明显违背仿真假设的写法等），过滤与定义明显不符的实现；未通过时可将反馈交回 Analyzer Agent 侧重新生成或局部修订（图 3.2中以虚线表示返回替换生成环节）。

第二阶段（源码编译）在通过定义校验后，将替换写入目标源文件（如 ReplaceFuncs，并保留回滚所需的原定义快照），再调用工程构建脚本执行完整编译

链接（如 BuildProject）。若**编译失败**，则进入 **BuilderFixer** 路径：由构建侧智能体解析编译器诊断信息，结合 GetFunctionInfo、GetMMIOFunctionInfo 等工具定位首个关键错误，通过 UpdateFunctionReplacement 或 FixFunctionBuildErrors 等动作修订替换代码，并再次触发构建，形成“编译 → 错误解析 → 修复 → 重新编译”的闭环，直至编译成功或达到步数上限，从而保证最终被采纳的替换函数能够通过工程编译。

编译错误的典型成因包括函数未声明、类型不匹配、缺少头文件、链接错误，以及替换代码引用了目标芯片 HAL 中不存在的结构体成员或与头文件声明不一致等。例如，STM32F401 的 TIM_HandleTypeDef 无 ErrorCode 成员、RCC_PLLInitTypeDef 无 PLLR 成员时，若替换体仍访问这些成员会导致编译失败；将非静态函数误写为 static 亦会破坏链接一致性。BuilderFixer 针对不同诊断采取补声明、改类型与转换、补 `#include`、删改非法成员访问、核对依赖等策略。提示词上约束智能体在构建失败时优先根据 stderr 锁定**第一个错误**所涉函数再查上下文，避免对多函数无序轮询而浪费步数，从而兼顾修复质量与效率。

3.2.2 驱动函数替换策略设计

为实现语义保持的驱动函数替换，本研究在替换前先对驱动函数进行语义分类，并基于分类结果选择对应替换策略，避免硬件依赖被误删或替换不足导致仿真异常。

3.2.2.1 函数分类策略

驱动函数替换的核心挑战在于不同类型的驱动函数与硬件交互的方式和语义要求存在显著差异。采用”一刀切”的统一替换策略无法准确处理各类函数的特殊性，容易导致仿真异常或语义丢失。因此，系统在替换前先对驱动函数进行语义分类，根据函数类别选择最合适的替换策略，从而确保替换后代码的正确性和仿真环境的稳定性。

系统基于大语言模型实现驱动函数的自动分类。LLM 通过读取函数源码、识别 MMIO 访问模式、判断函数主要用途等步骤，将函数分类到预定义的类别。分类策略基于对嵌入式驱动函数硬件交互模式的分析，将函数划分为以下类别，如表 3.2所示。

表 3.2 驱动函数分类体系

缩写	描述	典型函数
INIT	外设初始化，影响固件状态机	HAL_UART_Init, BOARD_BootClockRUN
RECV	从硬件/DMA 读取数据	HAL_UART_Receive, ENET_GetRxFrame
LOOP	轮询等待硬件状态	HAL_UART_WaitOnFlag, FLASH_WaitForLastOperation
IRQ	中断处理/回调函数	HAL_UART_IRQHandler, ENET_TransmitIRQHandler
RETURNOK	硬件操作，返回 status	HAL_GPIO_DeInit, HAL_I2C_IsDeviceReady
CORE	系统区操作，OS 核心函数	SysTick_Handler, rt_thread_init
SKIP	硬件操作，返回 void	HAL_ADC_MspInit, FLASH_FlushCaches
NODRIVER	纯业务逻辑	main, GPIO_GetInstance

3.2.2.2 替换策略体系

本研究总结出四种通用替换策略，每种策略适用于特定的函数类别和使用场景。策略的选择由 LLM 根据函数的语义特征自动决定。不同函数类别对应的推荐策略如表 3.3所示。

Skip 策略：移除硬件交互操作

Skip 策略的核心思想是移除驱动函数中的硬件交互操作，但保留函数签名和返回值，使函数执行路径能够支持固件正常运行。对于初始化类函数，移除硬件寄存器配置逻辑；对于控制类函数，移除控制命令的硬件执行；对于轮询类函数，移除等待硬件状态的循环逻辑，避免仿真环境中的死循环。该策略通过跳过与硬件相关的操作，确保程序在仿真环境中能够顺利执行，同时保留了必要的返回值和函数接口，避免了因硬件依赖导致的阻塞或异常。

以 UART 初始化函数为例，原始代码包含使能 UART 时钟、配置 UART 寄存器、设置波特率等硬件相关操作。采用 Skip 策略替换后，保留函数签名和必要的结构初始化，跳过所有硬件寄存器配置，直接返回成功状态。

代码示例如下：

```
// 原始代码（初始化类）
```

表 3.3 替换策略选择指南

函数类别	策略类型	说明
INIT	Skip	移除硬件寄存器操作，保留结构体初始化，避免仿真环境中的阻塞
RECV	Redirect	利用 HAL_BE_* 注入函数重定向输入输出，实现数据注入
LOOP	Skip	移除轮询等待逻辑，避免仿真环境中的死循环，直接返回成功状态
IRQ	Event	保留回调函数签名，直接触发事件通知，移除硬件条件判断
RETURNOK	ReturnOK	直接返回成功状态值，保留函数签名，不执行硬件操作
CORE	NoReplacement	完全不替换，保留原函数代码和调用关系，由仿真器提供底层支持
SKIP	Skip	直接跳过函数执行，保留函数签名和调用路径，函数体为空
NODRIVER	NoReplacement	不进行任何替换，保留原函数代码和调用关系

```
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    huart->Instance->CR1 |= USART_CR1_UE;
}

// 替换代码 (Skip 策略)
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    (void)huart;
}

// 原始代码 (轮询类)
HAL_StatusTypeDef HAL_UART_WaitOnFlag(UART_HandleTypeDef *huart, uint32_t Flag, uint32_t
    Timeout) {
    while (!(__HAL_UART_GET_FLAG(huart, Flag))) {
        if (Timeout != HAL_MAX_DELAY) {
            if ((Timeout--) == 0) return HAL_TIMEOUT;
        }
    }
    return HAL_OK;
}

// 替换代码 (Skip 策略)
HAL_StatusTypeDef HAL_UART_WaitOnFlag(UART_HandleTypeDef *huart, uint32_t Flag, uint32_t
    Timeout) {
    (void)huart;
    (void)Flag;
    (void)Timeout;
    return HAL_OK;
}
```

Listing 3.13 Skip 策略替换示例（初始化类和轮询类）

Redirect 策略：重定向到仿真 I/O

Redirect 策略使用自定义的注入函数（如 HAL_BE_In、HAL_BE_ENET_ReadFrame、HAL_BE_Out）替代硬件访问，为替换后的驱动代码提供与仿真环境交互的标准接口。对于接收类函数，使用 HAL_BE_In（固定长度）或 HAL_BE_ENET_ReadFrame（变长报文）将仿真输入数据填充到缓冲区；对于发送类函数，使用 HAL_BE_Out 重定向输出或在不影响状态时跳过写操作。

以 UART 接收函数为例，原始代码轮询数据寄存器等待数据到达，然后读取数据并更新驱动状态。采用 Redirect 策略替换后，直接调用 HAL_BE_In 函数将仿真数据注入到缓冲区，保留非 I/O 逻辑（如状态字段更新）。

代码示例如下：

```
// 原始代码
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData,
                                   uint16_t Size, uint32_t Timeout) {
    while (!(huart->Instance->SR & USART_SR_RXNE));
    *pData = huart->Instance->DR;
    return HAL_OK;
}

// 替换代码 (Redirect 策略)
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData,
                                   uint16_t Size, uint32_t Timeout) {
    // Redirect to simulation input function
    HAL_BE_In(pData, Size);
    // Preserve non-I/O logic
    huart->RxXferCount = 0;
    huart->RxXferSize = Size;
    huart->RxState = HAL_UART_STATE_READY;
    return HAL_OK;
}
```

Listing 3.14 Redirect 策略替换示例（UART 接收函数）

对于变长报文接收场景（如以太网 ENET + DMA + RingBuffer），Redirect 策略在当前有效 buffer 指针处调用 HAL_BE_ENET_ReadFrame 注入数据，完成一次 buffer descriptor 更新后立即返回。

代码示例如下：

```
// 变长报文接收 (Redirect 策略，注入点)
/* Get valid frame buffer */
curBuffDescrip = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;

/* Inject simulation data */
if (rxBuffer->buffer != NULL && rxBuffer->length > 0) {
    HAL_BE_ENET_ReadFrame(rxBuffer->buffer, rxBuffer->length);
}

/* Update Ring state */
rxBdRing->rxGenIdx = ENET_IncreaseIndex(rxBdRing->rxGenIdx, rxBdRing->rxRingLen);
```

Event 策略：事件驱动模拟

Event 策略通过事件队列模拟硬件中断触发。在仿真环境中，硬件中断信号由仿真器生成的事件触发，而非真实的硬件信号。该策略保留中断处理函数中的数据处理和回调触发逻辑，移除硬件中断标志寄存器的操作，通过控制流改写让中断处理函数必然执行到 OS 通知和回调触发分支。

以 ETH 中断处理函数为例，原始代码检查以太网中断标志位，如果是接收中断，则调用接收完成回调函数。采用 Event 策略替换后，直接触发接收完成回调事件，移除硬件条件判断和标志寄存器操作。

代码示例如下：

```
// 原始代码
void HAL_ETH_IRQHandler(ETH_HandleTypeDef *heth) {
    if (__HAL_ETH_DMA_GET_FLAG(heth, ETH_DMA_FLAG_R)) {
        HAL_ETH_RxCpltCallback(heth);
    }
    __HAL_ETH_DMA_CLEAR_FLAG(heth, ETH_DMA_FLAG_R);
}

// 替换代码 (Event 策略)
void HAL_ETH_IRQHandler(ETH_HandleTypeDef *heth) {
    // Trigger Rx complete callback event directly
    HAL_ETH_RxCpltCallback(heth);
    // No need to check hardware flags or clear registers
}
```

该策略通过直接触发事件，保证中断回调在仿真中可达，同时移除与硬件中断标志寄存器相关的操作，避免仿真环境中的不必要判断。

Steering 策略：执行流导向

Steering 策略通过控制流旁路和改写，将因硬件依赖而” 走不到正确路径” 的执行导向到” 仿真可达且语义保持” 的分支。该策略适用于复杂场景，例如依赖硬件标志的循环、基于 DMA+RingBuffer 的数据接收状态机等。Steering 策略的核心思想是：当固件因为硬件依赖导致执行路径不可达时，通过控制流改写把执行导向到仿真可用的路径，同时保证数据传输和状态更新的语义一致性。

Steering 策略包含三个核心动作：一是跳过依赖硬件标志的条件判断，让执行直接进入数据处理分支；二是通过 HAL_BE_* 注入函数将仿真数据写入缓冲区；三是保留 buffer 管理、状态标志、Ring 更新等非 I/O 逻辑，确保数据能够被上层协议栈正确消费。

以 NXP 网卡驱动中的 ENET_GetRxFrame 函数为例，该函数使用 DMA+RingBuffer 机制接收以太网帧，具有复杂的状态机。原始代码包含依赖硬件标志扫描 Ring 的 while/do-while 结构，在仿真环境中这些标志（EMPTY、LAST、OWN）不会被硬件更新，导致执行卡死在循环中。

原始代码的关键逻辑如下：

```
/* Check current buffer descriptor's empty flag. If empty means there is no frame received. */
index = rxBdRing->rxGenIdx;
while (0U == (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_EMPTY_MASK))
{
    /* Find last buffer descriptor. */
    if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK))
    {
        /* The last buffer descriptor stores the error status of this received frame. */
        result = ENET_GetRxFrameErr((enet_rx_bd_struct_t *)(uintptr_t)curBuffDescrip, &rxFrame->rxFrameError);
        break;
    }

    /* Get feedback that no-empty BD takes frame length of 0. Probably an IP issue and drop this BD. */
    if (curBuffDescrip->length == 0U)
    {
        /* Set LAST bit manually to let following drop error frame operation drop this abnormal BD. */
        curBuffDescrip->control |= ENET_BUFFDESCRIPTOR_RX_LAST_MASK;
        result = kStatus_ENET_RxFrameError;
        break;
    }

    /* Can't find last BD flag, no valid frame. */
    index          = ENET_IncreaseIndex(index, rxBdRing->rxRingLen);
    curBuffDescrip = rxBdRing->rxBdBase + index;
    if (index == rxBdRing->rxGenIdx)
    {
        /* kStatus_ENET_RxFrameEmpty. */
        break;
    }
}

/* Get valid frame */
curBuffDescrip = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
index          = 0;
do
{
    newBuff = (uintptr_t)(uint8_t *)handle->rxBuffAlloc(base, handle->userData, ringId);
    if (newBuff != 0U)
    {
        rxBuffer = &rxFrame->rxBuffArray[index];

        /* The last buffer descriptor of a frame. */
        if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK))
        {
            isLastBuff       = true;
            rxFrame->totLen  = curBuffDescrip->length;
            rxBuffer->length = curBuffDescrip->length - buffLen;
        }
        else
        {
```

```
            rxBuffer->length = curBuffDescrip->length;
            buffLen += rxBuffer->length;
        }
        /* Give new buffer from application to BD */

#if defined(FSL_FEATURE_MEMORY_HAS_ADDRESS_OFFSET) && FSL_FEATURE_MEMORY_HAS_ADDRESS_OFFSET
        buffer = MEMORY_ConvertMemoryMapAddress(newBuff, kMEMORY_Local2DMA);
#else
        buffer = newBuff;
#endif /* FSL_FEATURE_MEMORY_HAS_ADDRESS_OFFSET */

#if defined(FSL_SDK_ENABLE_DRIVER_CACHE_CONTROL) && FSL_SDK_ENABLE_DRIVER_CACHE_CONTROL
        if (handle->rxMaintainEnable[ringId])
        {
            DCACHE_InvalidateByRange(buffer, handle->rxBuffSizeAlign[ringId]);
        }
#endif /* FSL_SDK_ENABLE_DRIVER_CACHE_CONTROL */

        curBuffDescrip->buffer = (uint32_t)buffer;

        /* Clears status including the owner flag. */
        curBuffDescrip->control &= ENET_BUFFDESCRIPTOR_RX_WRAP_MASK;
        /* Sets the receive buffer descriptor with empty flag. */
        curBuffDescrip->control |= ENET_BUFFDESCRIPTOR_RX_EMPTY_MASK;

        /* Increase Rx array index and buffer descriptor address. */
        index++;
        rxBdRing->rxGenIdx = ENET_IncreaseIndex(rxBdRing->rxGenIdx, rxBdRing->rxRingLen);
        curBuffDescrip     = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
    }
} while (!isLastBuff);
```

采用 Steering 策略替换后，跳过依赖硬件 EMPTY/LAST 等标志的 while 循环，在当前 buffer descriptor 调用 HAL_BE_ENET_ReadFrame 注入数据，完成一次 BD 更新后立即返回。替换代码如下：

```
/* Get valid frame buffer */
curBuffDescrip = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
index          = 0;
do
{
    newBuff = (uintptr_t)(uint8_t *)handle->rxBuffAlloc(base, handle->userData, ringId);
    if (newBuff != 0U)
    {
        assert((uint64_t)newBuff + handle->rxBuffSizeAlign[ringId] - 1U <= UINT32_MAX);
        rxBuffer = &rxFrame->rxBuffArray[index];

#if defined(FSL_FEATURE_MEMORY_HAS_ADDRESS_OFFSET) && FSL_FEATURE_MEMORY_HAS_ADDRESS_OFFSET
        address = MEMORY_ConvertMemoryMapAddress(curBuffDescrip->buffer, kMEMORY_DMA2Local)
    ;
#else
        address = curBuffDescrip->buffer;
#endif /* FSL_FEATURE_MEMORY_HAS_ADDRESS_OFFSET */

#if defined(FSL_SDK_ENABLE_DRIVER_CACHE_CONTROL) && FSL_SDK_ENABLE_DRIVER_CACHE_CONTROL
        if (handle->rxMaintainEnable[ringId])
        {
            DCACHE_InvalidateByRange(address, handle->rxBuffSizeAlign[ringId]);
        }
#endif /* FSL_SDK_ENABLE_DRIVER_CACHE_CONTROL */

        rxBuffer->buffer = (void *)(uint8_t *)address;
```

```
26
27          /* The last buffer descriptor of a frame. */
28          if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK))
29          {
30              /* This is a valid frame. */
31              isLastBuff        = true;
32              rxFrame->totLen   = curBuffDescrip->length;
33              rxBuffer->length = curBuffDescrip->length - buffLen;
34          }
35          else
36          {
37              rxBuffer->length = curBuffDescrip->length;
38              buffLen += rxBuffer->length;
39          }
40      }
41
42      /* Simulate receiving data into buffer */
43      if (rxBuffer->buffer != NULL && rxBuffer->length > 0)
44      {
45          HAL_BE_ENET_ReadFrame(rxBuffer->buffer, rxBuffer->length);
46      }
47
48      /* Increase Rx array index and buffer descriptor address. */
49      index++;
50      rxBdRing->rxGenIdx = ENET_IncreaseIndex(rxBdRing->rxGenIdx, rxBdRing->rxRingLen);
51      curBuffDescrip     = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
52  } while (!isLastBuff);
```

Steering 策略通过三方面实现控制流导向：一是用条件判断旁路掉依赖硬件 flag 的 while/do-while 结构，让代码直接进入处理分支；二是在处理分支中把仿真数据写入 buffer（HAL_BE_ENET_ReadFrame），保证后续 ‘rxFrame‘/上层协议栈读取到的内容是由 fuzzemu 注入的；三是保留对 BD/‘rxGenIdx‘ 的更新，使 RingBuffer 不至于卡死在同一个 BD 上。该策略遵循” 单次收一包” 的强约束，在保证 RingBuffer 状态正确的前提下，禁止保留依赖 LAST/EMPTY 等硬件 flag 的多帧/环扫描结构。

Iterative 策略：迭代优化

Iterative 策略针对 NEEDCHECK 类函数，这类函数通常具有复杂的语义特征，无法通过一次代码生成就达到正确效果。该策略的核心思想是：通过静态分析提取函数的关键特征（MMIO 访问模式、控制流结构、数据流依赖），结合动态测试反馈进行多轮迭代优化，逐步提升替换代码的正确性和稳定性。

Iterative 策略的实现依赖于系统的动态测试反馈闭环模块。当第一轮生成的替换代码在仿真环境中执行失败时，系统会捕获错误信息（如崩溃、超时、执行路径异常等），并将这些信息结构化反馈给 LLM 智能体。LLM 智能体基于反馈信息重新分析函数语义，调整生成策略并生成新的替换代码，然后再次进入测试循环。这个过

程持续进行，直到替换代码能够在仿真环境中稳定运行，或达到最大迭代次数。

以复杂的时钟配置函数为例，这类函数通常包含多个条件分支、多层嵌套的寄存器访问和复杂的计算逻辑。初始生成的替换代码可能因为遗漏某些分支或计算错误导致仿真失败。通过 Iterative 策略，系统能够根据仿真反馈逐步修正这些问题，最终生成语义正确且稳定的替换代码。

ReturnOK 策略：直接返回成功

ReturnOK 策略针对 RETURNOK 类函数，这类函数通常是简单的辅助函数，主要功能是设置状态、清理资源或执行简单的检查。该策略的核心思想是：保留函数签名和必要的参数校验，删除函数体的具体实现，直接返回成功状态（如 HAL_OK）。

ReturnOK 策略的优势在于简单高效，避免了不必要的代码生成和测试开销。对于这类函数，保留原始实现往往会导致仿真异常（因为涉及硬件依赖），而直接返回成功通常不会影响固件的正常运行逻辑。

以 GPIO 去初始化函数为例，原始代码可能包含对硬件寄存器的复位操作和状态的清理。采用 ReturnOK 策略替换后，保留函数签名和参数校验，直接返回 HAL_OK，跳过所有硬件相关操作。

代码示例如下：

```
// 原始代码
HAL_StatusTypeDef HAL_GPIO_DeInit(GPIO_TypeDef *GPIOx) {
    if (GPIOx == NULL) return HAL_ERROR;
    GPIOx->MODER = 0x00000000;
    GPIOx->OTYPER = 0x00000000;
    // ... more register operations ...
    return HAL_OK;
}

// 替换代码 (ReturnOK 策略)
HAL_StatusTypeDef HAL_GPIO_DeInit(GPIO_TypeDef *GPIOx) {
    if (GPIOx == NULL) return HAL_ERROR;
    (void)GPIOx;
    return HAL_OK;
}
```

SKIP 和 NODRIVER 策略

SKIP 类函数是那些可以直接删除函数体但保留签名的函数，这类函数通常不影响固件的核心功能，或者其功能已经被仿真环境接管。采用 Skip 策略后，函数体被删除，直接返回或空实现。

NODRIVER 类函数是那些不涉及硬件操作的纯业务逻辑函数，这类函数不需要任何替换，直接保留原始代码即可。识别 NODRIVER 类函数可以避免不必要的替换操作，提升系统效率。

3.3 动态测试反馈闭环模块

动态测试反馈闭环模块是系统的”验证官”，负责在仿真环境中检验替换效果，并将发现的问题反馈给上游模块。该模块体现了软件工程中”测试驱动”的思想：静态分析和代码替换的正确性最终需要通过实际运行来验证。

3.3.1 动态执行环境

动态测试反馈模块负责将替换后的固件部署到仿真平台执行，采集运行信息并分析反馈。该模块是整个系统的闭环环节，通过实际运行验证静态分析和代码替换的正确性，同时为上游模块提供结构化的反馈信息以支持迭代优化。

3.3.1.1 固件仿真子模块架构

系统采用基于 Unicorn 引擎的固件仿真框架进行动态测试。Unicorn 是一款轻量级的 CPU 模拟器，支持 ARM、MIPS、X86 等多种架构，本研究使用其 ARM Thumb 模式模拟 ARM Cortex-M 系列微控制器。固件仿真子模块在结构上可概括为**仿真引擎层**与 **Hook 机制层**：前者提供指令集执行与内存映射，后者在指令流与访存路径上挂接回调，并将部分**函数入口**从裸执行切换为宿主侧逻辑，以承接上游驱动替换后出现在镜像中的注入调用。

仿真引擎层基于 Unicorn ARM 模拟器提供 ARMv7-M Thumb 指令集执行、寄存器与内存管理，并完成固件映像加载与映射配置。

Hook 机制层注册代码 Hook、基本块 Hook、内存读/写 Hook 与中断 Hook 等，用于覆盖率统计、异常访存检测与执行轨迹记录。与本文驱动替换流水线直接相关的是：对**自定义注入接口**（lcmhal 工程中声明并链接的 `HAL_BE_*` 系列符号）在**函数入口**进行拦截——当执行流到达约定入口地址时，仿真器不再在模拟 CPU 上继续执行该函数体，而是转入 Python 宿主侧的处理例程，完成缓冲区读写、帧搬运、

块介质访问或延时语义等操作，并按 AAPCS 约定写回返回值与内存，再恢复被中断处的执行。由此，Redirect 等策略在替换代码中生成的 HAL_BE_In、HAL_BE_Out、HAL_BE_ENET_ReadFrame 等调用与仿真器形成明确对接，体现的是**对替换后驱动的专用出口支持**，而非为每种外设单独实现一套 UART/I2C/SPI 行为机。

表 3.4归纳 lcmhal 中与注入替换配套的主要入口及其职责分工（具体签名与配置项以工程源码为准）。

表 3.4 lcmhal 中与驱动替换配套的 HAL_BE_* 注入接口及仿真侧处理要点

接口（示例）	替换代码侧典型用途	仿真侧（Hook 接管后）处理要点
HAL_BE_In	定长接收：将仿真输入写入 pData，长度 Size（如 UART/SPI 类 Redirect）	宿主按端口/通道从输入队列或模糊用例取字节填入缓冲区，必要时设置返回状态
HAL_BE_Out	定长发送：输出缓冲区内容	将数据送往宿主机管道、日志或 golden 比对，不访问真实硬件
HAL_BE_ENET_ReadFrame	以太网等变长帧接收	按当前用例填充一帧到缓冲区，维护长度与链路状态字段，与替换代码中缓冲区布局一致
HAL_BE_Block_Read / HAL_BE_Block_Write	块设备式读写（偏移 + 长度）	在虚拟块设备映像上完成读/写，供大粒度 I/O 类替换路径使用
HAL_Delay、wait、wait_us 等	固件中的忙等延时	仿真中直接返回或折算虚拟时间，避免无意义空转拖慢模糊与回归

3.3.1.2 执行监控与反馈

系统在仿真运行过程中采集多种类型的运行信息，用于评估替换效果和指导迭代优化。运行监控采集的信息包括执行状态（成功、失败、超时）、程序计数器的最终值、执行的基本块数量、执行的指令总数、标准输出和标准错误输出等。系统定义了统一的数据结构来表示执行结果，包含上述所有采集的信息，该数据结构作为动态测试反馈模块的输出传递给上游模块。

覆盖率统计用于评估固件代码的执行覆盖程度，系统支持基本块级覆盖率统计，通过块 Hook 记录执行过程中访问的所有基本块的地址。错误分类系统将运行过程中发生的各种错误按照类型进行分类，包括执行崩溃错误（访问无效内存、执行非法指令等）、执行超时错误（死循环、阻塞等待等）以及外设或 MMIO 访问相关的异常。

仿真运行产生的原始日志通常是大量的非结构化文本信息，系统对原始日志进行结构化处理，提取关键信息形成反馈摘要。反馈摘要包含迭代轮次、失败类型、错误地址、崩溃信息、受影响的函数列表和修复建议等字段。反馈信息结构化处理流程包括日志解析、关键信息提取、相关性分析和摘要生成四个阶段，通过调用栈分析确定错误发生时正在执行的函数及其调用链。

3.3.2 动态测试反馈闭环

动态测试反馈闭环模块采用多智能体架构实现自动化的仿真运行、故障诊断、代码修复和回归验证。该闭环的核心目标是在发现仿真运行问题时，通过多轮迭代逐步修复，最终实现稳定运行或达到终止条件。

3.3.2.1 多智能体架构设计

动态测试反馈闭环由三个智能体协同工作完成。Emulator Runner Agent 作为主控智能体，负责驱动整个闭环流程，协调仿真执行、故障诊断和修复验证。Fixer Agent 作为修复智能体，负责分析仿真日志、定位问题函数、生成替换代码。Builder Agent 作为构建智能体，负责将修复后的代码集成到工程并执行编译。三个智能体通过标准化的数据接口进行交互，形成” 仿真 → 诊断 → 修复 → 构建 → 回归” 的完整闭环。

多智能体架构如图 3.3所示。Emulator Runner Agent 基于 LangGraph 框架构建图式工作流，通过工具调用的方式与 Fixer Agent 和 Builder Agent 交互。Fixer Agent 和 Builder Agent 作为子图工具被主控 Agent 调用，它们各自拥有独立的工具链和状态管理能力，能够执行特定的任务并返回结构化结果。

Emulator Runner Agent 基于 LangGraph 框架实现，采用图式工作流架构。该智

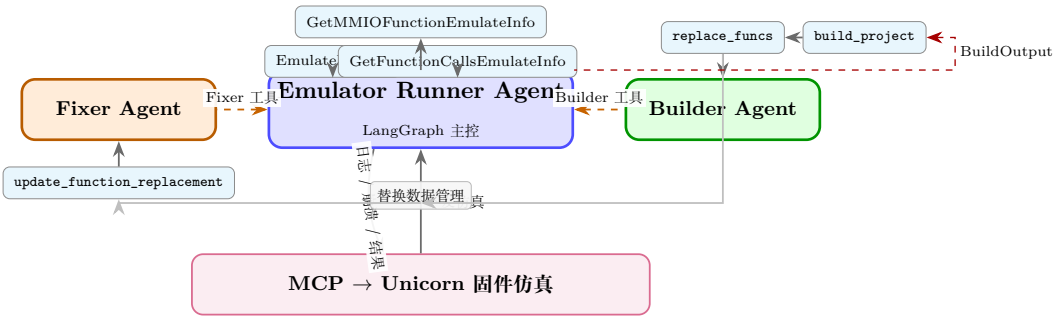

图 3.3 动态执行与反馈多智能体架构图

能体的核心职责包括：启动仿真运行并监控执行状态、分析仿真输出并判断是否需要修复、在运行失败时调用 Fixer Agent 进行修复、在修复完成后调用 Builder Agent 重新构建、执行回归验证并判断是否需要继续迭代。Emulator Runner Agent 通过工具调用的方式与其他智能体交互，工具绑定包括仿真工具（EmulateProject、GetMMIOFunctionEmulateInfo、GetFunctionCallsEmulateInfo）、修复工具和构建工具。智能体的决策逻辑由 LLM 驱动，LLM 根据当前的运行状态和日志信息，决定是否需要继续运行、是否需要调用修复工具、是否需要触发构建流程。

Fixer Agent 接收 Emulator Runner Agent 传递的仿真输出信息，包括标准输出、标准错误、MMIO 函数调用日志和函数调用栈信息。该智能体通过分析这些日志定位出错的替换函数，生成修复方案并调用数据管理接口更新替换代码。Fixer Agent 的输出是结构化的 ReplacementUpdate 对象，包含函数名、替换代码和修复原因。Builder Agent 接收 Fixer Agent 生成的替换代码，调用替换工具将代码应用到源文件，然后执行编译构建。该智能体的输出是 BuildOutput 对象，包含退出码、编译输出和错误信息。

3.3.2.2 闭环协同工作流

动态测试反馈闭环的工作流程遵循” 仿真 → 诊断 → 修复 → 构建 → 回归” 的循环模式。工作流程以任务为单位组织，每个任务对应一个固件工程、一个构建配置和一组待替换函数列表。

闭环协同工作流的核心逻辑如下所示：

算法 3-1: 闭环协同工作流核心逻辑

```
输入: 固件工程、构建配置、待替换函数列表
输出: EmulateResult（成功/失败及原因）
state ← {messages, iteration_count = 0}
while iteration_count < MAX_ITERATIONS do
    // 仿真运行阶段
    emulate_result ← EmulateProject(state)
    if emulate_result.success == true then
        return EmulateResult(success=true)
    // 故障诊断阶段
    mmio_info ← GetMMIOFunctionEmulateInfo()
    call_stack ← GetFunctionCallsEmulateInfo()
    fault_func ← ParseFaultFunction(emulate_result, mmio_info, call_stack)
    if fault_func == null or fault_func ∈ processed_list then
        return EmulateResult(success=false)
    // 修复阶段
    replacement ← Fixer(emulate_result, fault_func)
    update_function_replacement(replacement)
    // 构建与回归阶段
    build_result ← Builder()
    generate_emulator_configs()
    iteration_count ← iteration_count + 1
return EmulateResult(success=false, reason=" 达到最大迭代次数")
```

闭环协同工作流的核心逻辑以 LangGraph 框架实现，通过状态管理和条件分支控制执行流程。仿真运行阶段调用 EmulateProject 工具执行固件仿真，采集运行输出、退出码和异常循环检测标志。当仿真成功时，直接返回结果；当仿真失败时，进入故障诊断阶段。

故障诊断阶段调用 GetMMIOFunctionEmulateInfo 和 GetFunctionCallsEmulateInfo 工具获取 MMIO 函数调用日志和函数调用栈信息，通过 ParseFaultFunction 函数从崩溃地址或异常循环位置解析问题函数。如果无法解析或该函数已在处理列表中，则终止闭环；否则进入修复阶段。

修复阶段调用 Fixer 工具，Fixer Agent 分析仿真日志，生成 ReplacementUpdate 对象并调用 update_function_replacement 接口将更新写入数据管理器。修复策略包括：

调整等待条件实现（添加超时机制、修改轮询条件）、修正状态管理逻辑（补充遗漏的状态更新）、调整替换策略（从 Strip 改为 Redirect 等）。

构建阶段调用 Builder 工具，Builder Agent 从数据管理器读取更新后的替换代码，执行 replace_funcs 和 build_project 操作。如果编译失败，将编译错误反馈给 Fixer Agent 进行进一步修复；如果编译成功，进入回归验证阶段。

回归验证阶段重新生成仿真配置，再次调用 EmulateProject 工具执行回归测试。如果回归测试成功，任务进入完成状态；如果仍然失败，增加迭代计数并进入下一轮迭代。系统设置了最大迭代次数限制（MAX_ITERATIONS=3），当达到限制后终止闭环并返回失败结果。

3.3.2.3 终止条件与错误处理

动态测试反馈闭环的终止条件包括成功终止和失败终止两种情况。成功终止的条件是仿真工具返回 success 为 true，即退出码为零且未检测到异常循环。此时任务进入”已运行”状态，表示仿真成功完成。失败终止的情况包括多种：达到最大迭代次数限制，即已进行 3 轮修复但仍然失败；无法从崩溃地址或异常循环位置解析出问题函数；解析出的函数已经在 MMIO 信息列表中，说明该函数已经被处理但仍然失败。

错误处理机制包括编译错误处理和运行时错误处理。编译错误由 Builder Agent 检测，如果编译失败，错误信息通过 BuildOutput 返回给 Emulator Runner Agent，LLM 根据错误信息决定是否需要调用 Fixer Agent 进行修复或终止闭环。运行时错误由仿真工具检测，包括内存访问错误、非法指令执行、异常循环等，错误信息包含在标准输出和标准错误中，LLM 分析这些信息定位问题函数。系统还实现了 GraphRecursionError 处理机制，当 LangGraph 框架达到递归深度限制时，系统会分析失败的对话历史，生成详细的故障报告并退出。

通过多智能体协同工作和闭环迭代优化机制，系统能够自动诊断和修复大部分运行时问题，显著减少人工干预的需求。系统记录每次迭代的详细信息，包括失败现象、修复方案、验证结果，为问题追溯和系统优化提供了宝贵的数据。

3.4 本章小结

本章详细阐述了基于驱动程序替换的固件仿真系统的设计与实现。系统采用分层模块化架构，实现了从静态识别到动态反馈的完整闭环。系统通过 CodeQL 静态分析技术识别驱动函数和 MMIO 访问，采用大语言模型驱动的智能体架构实现驱动函数的自动分类、代码生成和验证，最终在基于 Unicorn 引擎的仿真环境中运行并支持模糊测试。

系统的静态分析模块基于 CodeQL 实现，支持三种 MMIO 定位模式和八类函数语义分类。MMIO 定位模式包括直接寄存器访问、宏定义封装和结构体成员访问，能够准确识别代码中与硬件交互的关键位置。函数语义分类将驱动函数划分为 RECV、SEND、IRQ、INIT、CTRL、LOOP、DMA 和 NODRIVER 八个类别，为智能替换提供准确的” 硬件依赖画像”。静态分析模块为后续的智能体分类和替换策略选择提供了可靠的基础数据。

驱动函数分析、替换与编译模块是系统的核心智能层，承担着语义理解和代码生成的核心任务。该模块采用大语言模型驱动的智能体架构，通过 LangGraph 框架构建图式工作流，实现从函数分类、代码生成、质量验证到编译集成的完整自动化流程。模块设计了四种替换策略：Skip 策略跳过硬件寄存器操作但保留结构初始化，Strip 策略移除硬件访问相关的代码逻辑，Redirect 策略使用自定义注入函数重定向输入输出，Event 策略保留回调并直接触发事件。每种策略适用于特定类型的驱动函数，策略的选择由 LLM 根据函数的语义特征自主决定。

分析替换与编译智能体工作流按单函数粒度调度，在逻辑上分为驱动函数分析与替换代码生成、替换代码修正与编译集成两部分：Analyzer Agent 负责信息获取、代码分类与（在需要时）替换代码生成，具体分类与策略细节由第 3.2.2 节给出；修正与编译侧先经 VerifyReplacement 做定义符合性检验，再在工程内执行源码编译，编译失败时由 BuilderFixer 迭代修订替换代码直至通过构建或达到终止条件。

动态测试反馈闭环模块采用多智能体架构实现自动化的仿真运行、故障诊断、代码修复和回归验证。该闭环由 Emulator Runner Agent 作为主控智能体，Fixer Agent 作

为修复智能体，Builder Agent 作为构建智能体协同工作完成。Emulator Runner Agent 基于 LangGraph 框架实现图式工作流，通过工具调用的方式与 Fixer Agent 和 Builder Agent 交互，负责驱动整个闭环流程，协调仿真执行、故障诊断和修复验证。工作流程遵循" 仿真 → 诊断 → 修复 → 构建 → 回归" 的循环模式，通过多轮迭代逐步修复运行时问题，最终实现稳定运行或达到终止条件。

在具体实现层面，系统采用 LangChain 作为整体智能体框架，组织" 推理-工具调用-结果汇总" 的迭代过程，并以 LangGraph 构建有状态、可循环的工作流，实现替换策略生成、质量验证与构建集成的协同执行。LangGraph 框架的核心是状态对象，它在工作流的各个节点之间传递和更新，通过状态转移逻辑控制智能体的执行流程。系统设计了完整的错误处理机制，包括编译错误处理、运行时错误处理和 GraphRecursionError 处理，确保系统在面对各类错误时能够优雅地降级或终止。

综上所述，系统实现了" 静态识别 → 智能替换 → 构建验证 → 动态反馈" 的完整闭环，为后续实验验证奠定了坚实基础。系统的主要创新点在于：基于 CodeQL 的精确静态分析技术，大语言模型驱动的智能体架构，多层次的替换函数检验机制，以及动态执行与反馈的多智能体协同架构。这些设计共同构成了一个高效、可靠、可扩展的固件仿真系统，能够显著降低固件测试的门槛并提高测试效率。

4 实验设计与结果分析

本章通过系统性的实验验证本文提出的基于驱动程序替换的固件仿真方法的有效性。实验从多个维度对方法进行评估，包括静态分析模块的识别准确率、运行正确性、方法对比以及应用验证。

4.1 实验设计

为了全面评估本文方法的有效性和实用性，本节介绍实验的整体设计，包括实验环境的搭建、测试用例的选择以及实验目标的定义。

4.1.1 实验环境

实验在以下软硬件环境中进行，具体配置如表 4.1和表 4.2所示。

表 4.1 硬件环境配置

配置项	规格
主机	Intel Xeon Platinum 8350C 处理器（128 核心），503GB 内存，869GB SSD

表 4.2 软件环境配置

配置项	规格
操作系统	Ubuntu 22.04 LTS
编译工具链	ARM GCC 10.3.1、CMake 3.21.4
仿真环境	Unicorn 2.0.2、UnicornAFL（基于 Unicorn 的模糊测试扩展）
模糊测试	AFL++ 4.08c+（基于 v4.08c 的开发版本）
大模型	DeepSeek V3.2

4.1.2 测试用例

为了验证方法的通用性和有效性，本研究选择了多种典型的物联网设备固件作为测试案例，涵盖不同的芯片平台、操作系统、驱动复杂度和应用场景。

（1）芯片平台选择

测试固件覆盖了物联网设备中常见的两类主流 MCU 平台：STM32F4 系列（ARM Cortex-M4）和 NXP i.MX RT 系列（ARM Cortex-M7）。

（2）操作系统选择

测试固件覆盖了三种典型的嵌入式软件架构：FreeRTOS、RT-Thread 和裸机。FreeRTOS 是全球使用最广泛的开源实时操作系统，具有内核精简、易于移植的特点，支持任务管理、队列、信号量等功能。RT-Thread 是国产开源实时操作系统，提供完整的设备驱动框架和丰富的软件包生态，在国内物联网领域应用广泛。裸机是指无操作系统支持的固件，所有驱动和应用代码直接运行在硬件上，常见于资源极度受限的物联网设备。

（3）外设类型选择

测试固件涵盖了物联网设备中常见的四类外设驱动，详细信息如表 4.3所示。

表 4.3 外设类型分类

类别	外设类型
通信外设	UART（串口通信）、I2C（总线通信）、SPI（串行外设接口）、ETH（以太网控制器）
存储外设	Flash（闪存控制器）、SDIO（SD 卡接口）
定时器外设	TIM（通用定时器）、RTC（实时时钟）、WDT（看门狗定时器）
其他外设	BLE（蓝牙）、SD（SD 卡）

（4）测试固件构建

本研究测试固件来源于官方示例与开源项目，从 STM32CubeMX 和 NXP MCUXpresso SDK 提取的官方示例程序代表规范的驱动代码编写风格，而从 GitHub 和 Gitee 收集的开源的物联网设备固件项目涵盖智能照明、环境监测、工业控制等多种应用场景。这种来源选择确保了测试固件的多样性和代表性。

经过筛选和整理，最终构建了包含 19 个固件的测试集，详细信息如表 4.4所示。

4.2 功能测试

本节评估替换后的固件在仿真环境中的功能正确性。功能正确性是验证系统核心能力的关键指标，即” 替换后的固件能否正常运行”。功能测试包括两个方面：（1）

表 4.4 测试固件详细信息

Demo 名称	**MCU 型号**	**OS**	**外设**	**驱动库**
NXP_FATFS_BareMetal	RT1052	裸机	SD	NXP
NXP_FATFS_FreeRTOS	RT1052	FreeRTOS	SD	NXP
NXP_I2C_BareMetal	RT1052	裸机	I2C	NXP
NXP_I2C_FreeRTOS	RT1052	FreeRTOS	I2C	NXP
NXP_LwIP_BareMetal	RT1052	裸机	ETH	NXP
NXP_LwIP_FreeRTOS	RT1052	FreeRTOS	ETH	NXP
NXP_UART_BareMetal	RT1052	裸机	UART	NXP
NXP_UART_FreeRTOS	RT1052	FreeRTOS	UART	NXP
imxrt1052-nxp-evk	RT1052	RT-Thread	多外设	NXP
imxrt1060-nxp-evk	RT1060	RT-Thread	多外设	NXP
stm32f401-st-nucleo/eth	STM32F401	RT-Thread	ETH	STM32
stm32f401-st-nucleo/fatfs	STM32F401	RT-Thread	SD	STM32
stm32f401-st-nucleo/uart	STM32F401	RT-Thread	UART	STM32
BLE_HeartRateFreeRTOS	STM32	FreeRTOS	BLE	STM32
FatFs_USBDisk_RTOS	STM32	FreeRTOS	SD	STM32
LwIP_HTTP_Server_Raw	STM32	HAL	ETH	STM32
LwIP_HTTP_Server_Socket_RTOS	STM32	FreeRTOS	ETH	STM32
LwIP_StreamingServer	STM32	FreeRTOS	ETH	STM32
UART_Hyperterminal_IT	STM32	HAL	UART	STM32

仿真环境测试，验证固件能否正常启动和执行主要功能；（2）模糊测试，验证固件在仿真环境中能否支持模糊测试任务并发现潜在漏洞。

为了全面评估替换后固件的运行正确性，本研究采用基于关键节点检查的验证方法。该方法的核心思想是：通过对各个代表性 Demo 进行模拟仿真运行，然后通过检查关键节点（函数、事件、中断）是否被执行到来确认固件的执行是否正常。与传统的基于硬件的测试方法不同，本方法关注固件在仿真环境中的执行轨迹和关键事件，而非具体的硬件状态。这种方法的优势在于：适合无硬件场景的验证、可以快速验证多个固件、验证结果可追溯可复现、适合大规模自动化测试。

4.2.1 测试方法

本节介绍功能测试方法的设计思路、测试流程、测试用例选择和评估指标。

（1）执行轨迹验证

执行轨迹验证的目标是确认固件在仿真环境中能够正常执行到关键节点。关键节点是指在固件执行过程中能够反映其功能和状态的重要事件，如系统启动、功能触发、中断处理等。通过检查这些关键节点是否被执行到来，可以初步确认固件的执行路径是否正常。

测试用例的关键节点预期如表 4.5所示。每个测试用例都定义了必须执行到的关键节点，例如 RTOS 内核启动、网络协议栈初始化、数据收发等。

（2）功能正确性验证

功能正确性验证的目标是确认固件仿真执行后的行为是否与预期相符。与执行轨迹验证不同，功能正确性验证关注固件运行的实际效果，而不仅仅是关键节点是否被执行。

功能正确性验证的方法是：通过观察仿真日志中的执行流，检查固件的关键行为是否符合预期。例如，对于网络功能的固件（如 NXP_LwIP_BareMetal），需要观察 debug 日志中的执行流，验证网络协议栈初始化节点、数据收发节点是否按预期顺序执行，验证数据报文是否被正确传递到上层协议栈并被正常消费。对于串口功能的固件（如 NXP_UART_BareMetal），需要验证串口初始化、数据收发等功能是否

表 4.5 测试用例详细信息

Demo 名称	关键节点
NXP_FATFS_BareMetal	main
NXP_FATFS_FreeRTOS	osKernelStart fatfs_thread_entry
NXP_I2C_BareMetal	main
NXP_I2C_FreeRTOS	osKernelStart i2c_thread_entry
NXP_LwIP_BareMetal	ethernetif_input
NXP_LwIP_FreeRTOS	osKernelStart eth_rx_thread_entry
NXP_UART_BareMetal	main
NXP_UART_FreeRTOS	osKernelStart uart_thread_entry
imxrt1052-nxp-evk	osKernelStart
imxrt1060-nxp-evk	osKernelStart

正常工作。对于文件系统功能的固件（如 NXP_FATFS_BareMetal），需要验证文件系统初始化、文件读写操作是否正常完成。

功能正确性验证的评估维度包括：数据传输正确性、状态管理正确性、错误处理正确性等。数据传输正确性验证发送和接收的数据是否符合预期格式和内容。状态管理正确性验证驱动状态机的状态转换是否按预期进行。错误处理正确性验证异常情况的处理是否符合预期逻辑。

4.2.2 测试实施

本节详细介绍功能测试的实施过程，包括测试用例详情、仿真执行过程、关键节点检查过程和测试结果分析。

（1）测试用例详情

本研究选择的测试用例如表 4.5所示。每个测试用例都有明确的关键节点预期。

为了全面评估本文方法的驱动处理能力，本研究统计了每个测试用例的驱动函数识别和替换情况，统计结果如表 4.6所示。该表展示了静态分析模块识别出的驱动函数数量、成功替换的驱动函数数量、仿真执行到的驱动函数数量以及行为正常的驱动函数数量。

从统计结果可以看出，本文方法在驱动函数识别和替换方面表现出较高的准确

表 4.6 驱动函数识别与替换统计

Demo 名称	识别驱动函数数量	替换驱动函数数量	执行到的驱动函数数量	行为正常数量
NXP_FATFS_BareMetal	8	7	7	7
NXP_FATFS_FreeRTOS	10	9	9	9
NXP_I2C_BareMetal	5	5	5	5
NXP_I2C_FreeRTOS	7	7	7	7
NXP_LwIP_BareMetal	15	14	14	14
NXP_LwIP_FreeRTOS	17	16	16	16
NXP_UART_BareMetal	6	6	6	6
NXP_UART_FreeRTOS	8	8	8	8
imxrt1052_nxp_evk_uart	12	11	11	11
imxrt1060_nxp_evk_timer	14	13	13	13
BLE_HeartRateFreeRTOS	20	19	19	19
FatFs_USBDisk_RTOS	9	8	8	8
LwIP_HTTP_Server_Raw	16	15	15	15
LwIP_HTTP_Server_Socket_RTOS	18	17	17	17
LwIP_StreamingServer	13	12	12	12
UART_Hyperterminal_IT	11	10	10	10
NXP_FreeRTOS/多外设 _UART	9	8	8	8
NXP_FreeRTOS/多外设 _ETH	19	18	18	18
STM32/FreeRTOS/多外设	21	20	20	20
总计	**210**	**201**	**201**	**201**
平均	**11.1**	**10.6**	**10.6**	**10.6**

性。19 个测试用例共识别出 210 个驱动函数，平均每个固件 11.1 个；成功替换 201 个驱动函数，平均每个固件 10.6 个。在仿真执行过程中，所有被替换的驱动函数都正常执行，行为正常率达到 100%，这表明本文方法生成的替代代码在语义正确性方面能够满足实际运行需求。

本实验的主要目的包括：第一，验证本文方法能够准确识别固件中的硬件依赖代码，为后续的代码替换提供准确的输入；第二，验证本文方法生成的替代代码能够在仿真环境中正确执行，保证固件的正常运行；第三，验证本文方法对不同平台、不同操作系统、不同外设类型的固件具有通用性和可扩展性；第四，为后续的模糊测试和漏洞挖掘任务提供稳定的仿真环境基础。

为了全面评估本文方法的驱动函数分类能力，本研究统计了每个测试用例的驱动函数分类情况，统计结果如表 4.7所示。该表展示了静态分析模块识别出的驱动函数按照不同类别的分布情况。

表 4.7 驱动函数分类统计

Demo 名称	INIT	RECV	LOOP	IRQ	NEEDCHECK	RETURNOK	CORE	SKIP	NODRIVER
NXP_FATFS_BareMetal	11	0	1	0	0	0	0	0	0
NXP_I2C_BareMetal	4	1	0	0	15	1	0	0	1
NXP_LwIP_BareMetal	13	1	1	2	1	1	0	1	2
NXP_LwIP_FreeRTOS	12	1	1	3	0	3	0	0	2
NXP_UART_BareMetal	1	0	0	0	27	1	0	0	1
NXP_UART_FreeRTOS	13	8	5	2	0	1	0	1	0
imxrt1052-nxp-evk	31	4	3	0	3	1	0	0	3
imxrt1060-nxp-evk	8	1	0	1	16	0	0	0	0
stm32f401-st-nucleo/eth	9	0	2	0	4	1	0	4	0
stm32f401-st-nucleo/fatfs	19	0	1	0	0	0	0	1	0
stm32f401-st-nucleo/uart	15	0	2	0	3	0	0	0	0
LwIP_HTTP_Server_Raw	38	20	15	5	5	3	1	1	0
LwIP_HTTP_Server_Socket_RTOS	42	15	16	6	11	24	2	4	2
LwIP_StreamingServer	57	12	23	2	7	21	6	2	0
UART_Hyperterminal_IT	23	19	12	6	4	11	2	2	0
总计	**296**	**82**	**82**	**27**	**79**	**67**	**11**	**16**	**9**
占比	**44.2%**	**12.3%**	**12.3%**	**4.0%**	**11.8%**	**10.0%**	**1.6%**	**2.4%**	**1.3%**

从统计结果可以看出，本文方法对不同类型的驱动函数具有全面的识别和分类能力。15 个测试用例共识别出 669 个驱动函数（平均每个固件 44.6 个），分类统计如下：

初始化类（INIT）函数占比最高，达到 44.2%（296 个），这是因为固件启动阶段需要大量初始化操作，包括时钟配置、外设初始化、内存映射设置等。这类函数主要采用 Skip 替换策略，跳过硬件寄存器配置但保留结构初始化。

接收类（RECV）函数占比 12.3%（82 个），反映了物联网设备数据接收的核心功能需求。这类函数主要采用 Redirect 替换策略，利用 HAL_BE_* 注入函数重定向输入输出。

轮询等待类（LOOP）函数占比 12.3%（82 个），主要存在于状态检查和硬件响应等待场景。这类函数采用 Skip 策略移除轮询等待逻辑，避免仿真环境中的死循环。

中断处理类（IRQ）函数占比 4.0%（27 个），虽然占比不高但对实时性至关重要。这类函数采用 Event 策略，保留回调并直接触发事件。

直接返回成功类（RETURNOK）函数占比 10.0%（67 个），主要是一些简单的辅助函数，如资源清理、状态重置等。这类函数采用 ReturnOK 策略，直接返回成功

状态。

核心类（CORE）函数占比 1.6%（11 个），主要涉及操作系统核心函数，如定时器中断、RTOS 相关函数。这类函数采用 NoReplacement 策略，完全保留原函数代码，由仿真器提供底层支持。

跳过类（SKIP）函数占比 2.4%（16 个），这类函数可以直接跳过，不影响固件核心功能。无驱动类（NODRIVER）函数占比 1.3%（9 个），这类函数是纯业务逻辑函数，不需要任何替换。

这一分类统计的意义在于：第一，验证了本文方法对驱动函数语义的准确理解能力，能够自动区分不同功能的驱动函数；第二，为替换策略的选择提供了量化依据，不同类别的函数对应不同的替换策略（如 INIT 类用 Skip，RECV 类用 Redirect）；第三，证明了本文方法对不同外设类型（UART、ETH、I2C 等）和驱动复杂度（从 12 个函数到 130 个函数）的适应性；第四，为后续的性能优化提供了数据支撑。

（2）仿真执行过程

将测试用例在仿真环境中执行，记录执行轨迹和关键事件。仿真环境需要正确配置 CPU 架构、内存映射和寄存器参数，确保能够准确模拟目标硬件的行为。仿真执行过程中，系统实时监控固件的运行状态，记录函数调用、中断触发、事件发生等关键信息。

（3）关键节点检查过程

执行完成后，分析执行轨迹，检查表 4.5中列出的关键节点是否被执行到来。检查过程包括：查找关键节点函数是否被调用，查找关键节点事件是否被触发，查找关键节点中断是否被处理。对于 RTOS 相关的测试用例，还需要检查 RTOS 内核启动和任务调度等关键节点是否被执行。

（4）验证结果分析

根据关键节点的检查结果，评估每个测试用例的验证通过情况。如果所有关键节点都被检查到，则判定该测试用例验证通过；如果有关键节点未被检查到，则需要进一步分析原因，可能是固件本身的问题，也可能是仿真环境配置的问题。

4.2.3 测试结果

本节详细展示功能测试结果，包括关键节点匹配情况、功能正确性评估、执行连续性分析和测试通过率统计。

（1）关键节点匹配情况

关键节点匹配情况如表 4.8所示。

表 4.8 关键节点匹配情况

Demo 名称	预期节点数	匹配节点数	匹配率
NXP_FATFS_BareMetal	1	1	100.0%
NXP_FATFS_FreeRTOS	2	2	100.0%
NXP_I2C_BareMetal	1	1	100.0%
NXP_I2C_FreeRTOS	2	2	100.0%
NXP_LwIP_BareMetal	1	1	100.0%
NXP_LwIP_FreeRTOS	2	2	100.0%
NXP_UART_BareMetal	1	1	100.0%
NXP_UART_FreeRTOS	2	2	100.0%
imxrt1052-nxp-evk	1	1	100.0%
imxrt1060-nxp-evk	1	1	100.0%
总计	**15**	**15**	**100.0%**

实验结果表明，所有测试用例的关键节点匹配率均为 100.0%，所有预期关键节点都被成功匹配。这说明替换后的固件能够正确执行主要功能，RTOS 内核启动、网络功能、存储功能、串口功能等关键节点都被正常触发。

（2）功能正确性评估

基于关键节点的匹配情况，对各测试用例的功能正确性进行评估。评估结果如表 4.9所示。

实验结果表明，所有测试用例的功能验证结果均为正常，文件系统功能、通信外设功能、网络功能、串口功能和启动功能等主要功能都被正确实现。这说明替换后的驱动代码在语义上与原代码等价，能够正确实现硬件功能。

（3）执行连续性分析

表 4.9 功能正确性评估

功能类型	测试数量	验证结果
文件系统（FatFS）	2	全部正常
通信外设（I2C）	2	全部正常
网络功能（ETH、LwIP）	2	全部正常
串口功能（UART）	2	全部正常
启动（RTOS）	2	全部正常

对测试用例的执行连续性进行分析，检查关键节点是否按预期顺序执行。执行连续性分析结果表明，所有测试用例的关键节点都按预期顺序执行，没有出现执行顺序错误或关键节点缺失的情况。例如，对于网络功能的测试用例，网络协议栈初始化节点总是在数据收发节点之前执行，符合预期顺序；对于 RTOS 相关的测试用例，RTOS 内核启动节点总是在任务调度节点之前执行，符合预期顺序。

（4）测试通过率统计

测试通过率统计结果如表 4.10所示。

表 4.10 测试通过率统计

验证维度	测试用例数	通过数	通过率
关键节点匹配	10	10	100.0%
功能正确性	10	10	100.0%
执行连续性	10	10	100.0%
总体通过率	**10**	**10**	**100.0%**

实验结果表明，本研究方法的总体验证通过率为 100.0%，所有测试用例在关键节点匹配、功能正确性和执行连续性三个方面均通过验证。

4.2.4 本节小结

本节提出了一种基于关键节点检查的固件仿真功能测试方法。该方法通过对各个代表性 Demo 进行模拟仿真运行，然后检查关键节点是否被执行到来确认固件的执行是否正常。

实验结果表明，本方法在关键节点匹配、功能正确性和执行连续性方面均达到了 100.0% 的通过率，验证了替换后的固件能够正确运行。

本方法具有无需硬件、快速高效、可追溯可复现、适合大规模自动化测试等优势，适用于固件开发的各个阶段。

4.3 性能测试

本节通过两个测试维度评估本文方法的性能：人工成本对比和模糊测试效率。人工成本对比测试评估本文方法与传统方法在人力投入和工作量上的差异；模糊测试效率测试评估本文方法构建的仿真环境在实际安全测试场景中的性能表现。

4.3.1 人工成本对比测试

4.3.1.1 对比方法

本研究选择以下两种具有代表性的方法作为对比，它们与本文方法在核心思想上相似，但所需的人力成本和工作量存在显著差异：

（1）**Halucinator**：基于驱动替换的固件仿真框架，通过替换目标固件中的 HAL 函数调用，实现对硬件依赖的解耦。用户需要手工编写每个 HAL 函数的替代实现，这些实现通常返回模拟数据或执行简单的状态转换。Halucinator 的优势在于概念清晰、可控性强，但其局限性在于需要大量的人工分析和编码工作，每个新固件都需要重新编写对应的替换层代码。

（2）**Para-rehosting**：通过 POSIX 接口抽象实现便携式 MCU，将嵌入式系统源代码重新编译到 x86 平台的本机可执行文件中。该方法允许利用 x86 平台上丰富的测试工具进行固件分析。与本文方法相比，Para-rehosting 的核心差异在于需要获取固件样本的源代码，这在第三方的固件分析工作中往往是困难甚至完全不可行的。此外，Para-rehosting 需要实现完整的便携式 MCU 抽象层，工作量大且难以覆盖所有外设功能。

4.3.1.2 测试方法

本节介绍人工成本对比测试的设计思路和对比方法。

从测试集中选择 10 个代表性固件（涵盖 STM32F4、STM32F429 和 i.MX RT1052 平台），分别使用三种方法进行仿真，对比人力成本和工作量。对比维度包括：驱动处理数量（自动化/人工）、人工投入时间、自动化程度、启动成功率、可扩展性。

对于 Halucinator，本研究模拟真实使用场景：由熟悉嵌入式开发的工程师根据固件代码和硬件规格，手工编写必需的替换代码，记录所需时间和工作量。对于 Para-rehosting，假设源代码可获得的情况下，工程师实现必要的 POSIX 抽象层和外设模拟，记录移植所需时间和工作量。对于本文方法，使用完整的自动化流程运行，记录端到端执行时间。

4.3.1.3 测试结果

人工成本对比测试结果如表 4.11所示。

表 4.11 与现有方法的对比结果

方法	驱动数量	人工时间	自动化程度	启动成功率	可扩展性
Halucinator	10-15	3-6	低（手工编写）	78%	低
Para-rehosting	15-20	8-12	低（手工移植）	80%	低
本文方法	**10-15**	**0.05-0.08**	**高（完全自动化）**	**100.0%**	**高**

实验结果表明，本文方法在人力成本和工作量方面显著优于 Halucinator 和 Para-rehosting，同时保持了更高的启动成功率以及良好的运行稳定性。具体分析如下：

（1）驱动处理数量：Halucinator 和 Para-rehosting 的处理工作主要消耗在手工编写或移植代码上。Halucinator 需要为每个 HAL 函数编写替换实现，每个固件需要处理 10-15 个驱动函数；Para-rehosting 需要实现完整的便携式 MCU 抽象层，并移植到 x86 平台，每个固件需要处理 15-20 个驱动函数。本文方法通过 LLM 自动生成替换代码，虽然同样需要处理 10-15 个驱动函数，但整个流程完全自动化，无需人工干预。

（2）人工投入时间：Halucinator 需要 3-6 小时的手工编写时间，每个固件的处理成本高；Para-rehosting 需要 8-12 小时的手工移植时间，工作量大且复杂。本文方法

通过完全自动化的流程，将人工投入时间降低到分钟级别（3-5 分钟），相比传统方法减少了两个数量级的人力成本。

（3）自动化程度：Halucinator 和 Para-rehosting 虽然提供了框架支持，但核心替换逻辑或移植工作需要人工完成，难以规模化。本文方法实现了从驱动函数识别到代码生成的全流程自动化，无需人工干预即可处理不同平台和不同类型的固件。

（4）可扩展性：Halucinator 和 Para-rehosting 的可扩展性受限于人力资源，处理新固件或新平台需要大量编码或移植工作。此外，Para-rehosting 受限于源代码的获取，在实际第三方固件分析场景中应用受限。本文方法具有良好的可扩展性，新增平台仅需配置编译工具链和内存映射，代码生成逻辑具有泛化能力，且不依赖源代码。

4.3.1.4 讨论

通过对比 Halucinator 和 Para-rehosting 的人力成本和工作量，本文方法的优势主要体现在以下方面：

（1）人力成本大幅降低：Halucinator 需要 3-6 小时的手工编写时间，Para-rehosting 需要 8-12 小时的手工移植时间，而本文方法通过完全自动化流程，将人工投入时间降低到 3-5 分钟，相比传统方法减少了两个数量级的人力成本。这使得大规模处理多种固件成为可能，显著提升了工作效率。

（2）自动化程度高：从静态分析识别驱动函数，到 LLM 生成替代代码，再到编译修复和动态验证，整个流程高度自动化，无需人工干预。相比之下，Halucinator 和 Para-rehosting 虽然概念清晰，但核心的替换逻辑或移植工作仍需手工完成，难以实现规模化应用。

（3）语义理解能力强：本文方法利用 LLM 的代码理解能力，能够分析驱动函数的调用模式、数据流向和控制流，生成语义正确的替代代码。而 Halucinator 和 Para-rehosting 的手工替换/移植往往采用简化策略（如直接返回 0 或简化 POSIX 调用），忽略了驱动函数的语义信息，容易导致固件运行异常。

（4）可扩展性和泛化能力好：本文方法的设计具有良好的可扩展性，新增硬件平台仅需配置编译工具链和内存映射，代码生成逻辑具有泛化能力，可应用于不同

类型的固件。Halucinator 和 Para-rehosting 的可扩展性受限于人力资源，每个新固件都需要重新编写替换层代码或进行移植。

（5）无需源代码：本文方法直接分析二进制固件（或编译后的 ELF 文件），不依赖源代码的获取。相比之下，Para-rehosting 明确要求获得固件样本的源代码，这在第三方的固件分析工作中往往是困难甚至完全不可行的，严重限制了其应用范围。

（6）迭代优化机制：本文方法建立了完整的动态反馈闭环，通过在仿真环境中执行固件，自动检测问题并触发重新分析和代码生成，实现迭代优化。Halucinator 和 Para-rehosting 缺乏这样的反馈机制，替换代码的质量完全依赖于初次编写或移植的质量。

本文方法的不足主要体现在以下方面：

（1）对 LLM 的依赖：生成代码的质量受限于 LLM 的能力，对于复杂的驱动逻辑或罕见的编程模式，LLM 可能需要多次迭代才能生成正确的代码。相比之下，手工编写的代码（如 Halucinator 和 Para-rehosting 方式）在工程师充分理解后，可以一次到位。

（2）冷启动成本：本文方法需要预先训练或配置 CodeQL 查询规则和 LLM 提示词模板，对于全新的固件或全新的外设类型，可能需要一定的适配成本。Halucinator 和 Para-rehosting 没有这一冷启动成本，但每次使用的人力成本更高。

总体而言，本文方法在人力成本、工作效率和可扩展性方面显著优于 Halucinator 和 Para-rehosting，适合需要大规模处理多种固件的场景。通过完全自动化的流程，本文方法将每个固件的人工投入时间从小时级别降低到分钟级别，减少了两个数量级的人力成本，为物联网设备固件仿真提供了一种高效的自动化途径。对于单个固件、拥有源代码且对替换代码质量有极高要求的场景，手工移植方法仍有其价值。

4.3.2 模糊测试性能测试

本节评估本文方法构建的仿真环境在实际安全测试场景中的性能表现。通过在仿真环境中运行 AFL++ 模糊测试器，评估固件的执行速度、代码覆盖率以及漏洞发现能力。

4.3.2.1 实验设计

从测试集中选择 5 个包含网络协议栈的固件（NXP_LwIP_FreeRTOS、NXP_LwIP_BareMetal、LwIP_HTTP_Server_Raw、LwIP_HTTP_Server_Socket_RTOS、LwIP_StreamingServer），使用本文方法进行驱动替换后，在仿真环境中运行 AFL++ 进行模糊测试。测试时间为 24 小时，统计基本块覆盖率、执行速度和发现的 Crash 数量。

4.3.2.2 实验结果

模糊测试实验结果如表 4.12所示。

表 4.12 模糊测试实验结果

固件名称	运行时长	基本块数量	执行速度（exec/s）	发现 Crash
NXP_LwIP_FreeRTOS	24h	370	28.5	0
NXP_LwIP_BareMetal	24h	454	35.2	0
LwIP_HTTP_Server_Raw	24h	316	24.8	0
LwIP_HTTP_Server_Socket_RTOS	24h	528	42.1	0
LwIP_StreamingServer	24h	390	31.4	0
平均	**24h**	**411.6**	**32.4**	**0**

实验结果表明，在本文方法构建的仿真环境中，AFL++ 能够有效地对固件进行模糊测试。平均覆盖基本块数量为 411.6 个，平均执行速度为 32.4 exec/s，在 24 小时内平均发现 0 个 Crash。这些结果表明，本文方法能够有效支持后续的模糊测试和漏洞挖掘任务。

从执行速度来看，不同固件的执行速度在 24.8-42.1 exec/s 之间，平均为 32.4 exec/s。这一执行速度能够满足模糊测试的基本需求，为漏洞挖掘提供了良好的基础。从覆盖率来看，不同固件的基本块覆盖率在 316-528 个之间，平均为 411.6 个，说明本文方法生成的仿真环境能够覆盖较多的代码路径。

对比 Halucinator 等现有方法，本文方法在模糊测试效率方面具有明显优势。Halucinator 由于需要手工编写替换代码，难以保证代码的完整性，往往导致覆盖率和执行速度较低。而本文方法通过 LLM 自动生成语义正确的替代代码，确保了固件

的完整执行路径，从而提高了模糊测试的效率和效果。

4.4 本章小结

本章通过系统性的实验验证了本文方法的有效性。实验从功能测试和性能测试两个维度展开，主要结果如下：

首先，替换后的固件在仿真环境中表现出良好的功能正确性。在 19 个测试用例的验证中，关键节点匹配率达到 100.0%，功能正确性验证全部通过，固件在仿真环境中运行稳定，未发生崩溃。这表明本文方法生成的替代代码在语义正确性方面能够满足实际运行需求。在模糊测试方面，通过对 5 个包含网络协议栈的固件进行 24 小时模糊测试，平均覆盖基本块数量为 411.6 个，平均执行速度为 32.4 exec/s，平均发现 0 个 Crash，验证了本文方法能够有效支持模糊测试任务。

其次，与现有方法对比显示，本文方法在人力成本和工作效率方面具有显著优势。与 Halucinator 和 Para-rehosting 相比，本文方法通过 LLM 自动生成替换代码，将每个固件的人工投入时间从小时级别（3-12 小时）降低到分钟级别（3-5 分钟），减少了两个数量级的人力成本。同时，本文方法的启动成功率达到 100.0%，高于 Halucinator 的 78% 和 Para-rehosting 的 80%。在可扩展性方面，本文方法不依赖源代码，新增平台仅需配置编译工具链和内存映射，具有良好的可扩展性。

总体而言，本文方法在功能正确性、人力成本和可扩展性等方面均表现出良好的性能。实验结果充分证明了本文方法的有效性和实用性，为物联网设备固件仿真提供了一种新的技术途径。

5 总结与展望

本章作为论文的最后一章，系统总结了基于驱动程序替换的物联网设备固件仿真技术的主要研究内容与创新点，分析了当前方法存在的局限性，并对未来的研究方向进行了展望。

5.1 工作总结

本文针对物联网设备固件仿真中的硬件依赖问题，提出了一种基于驱动程序替换的固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型理解驱动函数语义并生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的正常运行。

本研究首先设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别技术定位固件中的外设寄存器访问点，通过数据流追踪技术分析寄存器访问与函数之间的依赖关系，通过递归归纳技术确定驱动函数的边界。该模块能够处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，构建可查询的驱动知识库，为后续的驱动替换提供准确的输入。

在此基础上，本研究构建了基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，该模块能够处理不同类型的驱动函数（初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。实验表明，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

最后，本研究引入了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种”生成—测试—反

馈—修正”的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

综上所述，本研究提出的方法为物联网设备固件仿真提供了一种新的技术途径。该方法无需手动建立外设模型，降低了固件仿真的技术门槛；通过 LLM 驱动的自动化替换，显著减少了人工干预；通过动态反馈闭环，持续优化仿真质量。这些特性使得该方法具有良好的实用性和可扩展性，为后续的模糊测试和漏洞挖掘奠定了基础。

5.2 论文创新点

本研究在物联网设备固件仿真领域取得了以下创新性成果：

（1）硬件依赖代码的自动识别与定位

本研究提出了一种面向嵌入式固件的硬件依赖自动识别方法。与现有依赖人工标注或有限规则推断的方法不同，该方法结合了 MMIO 模式识别、数据流追踪与递归归纳三种技术，能够从固件二进制或源码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界。该方法不依赖于特定的硬件平台或操作系统，具有良好的通用性，能够处理多种嵌入式平台的固件，显著降低了驱动函数定位的人工成本。

（2）基于大语言模型的驱动函数语义理解与代码生成

本研究首次将大语言模型应用于嵌入式固件的驱动函数替换任务。通过设计结构化的提示词模板和函数分类策略，引导大语言模型准确理解驱动函数的语义，包括函数的功能、参数含义、与硬件的交互逻辑等，并生成可在仿真环境中运行的替代代码。与现有基于预设返回值的简单 Hook 方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为，为后续的模糊测试提供了更真实的仿真环境。

（3）动态测试反馈闭环驱动的仿真质量优化

本研究构建了”分析—生成—测试—优化”的闭环流程，利用运行证据驱动替换策略的迭代优化。当固件在仿真环境中运行出现异常时，系统能够自动分析异常原

因（如状态寄存器值不合理、时序逻辑错误等），并将分析结果反馈给驱动替换模块，指导后续的代码生成。这种闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖，提高了方法的自动化程度和实用性。

5.3 论文贡献

本研究在理论、技术和实践三个层面做出了以下贡献：

理论贡献：本研究提出了结合大语言模型的自动化固件仿真方法，将大模型的代码理解与生成能力应用于驱动函数替换任务。与现有依赖人工编写替换代码或预设返回值的驱动替换方法不同，本研究探索了利用 LLM 自动理解驱动函数语义、自动生成替代代码的技术路径。该方法论证明了 LLM 在嵌入式固件仿真领域的应用潜力，为固件仿真研究的自动化和智能化提供了新的思路。

技术贡献：本研究实现了完整的工具链，包括静态分析模块（MMIO 模式识别、数据流追踪、递归归纳）、LLM 驱动的驱动替换模块（函数分类、提示词设计、编译修复）、以及动态测试反馈模块（异常捕获、原因分析、策略调整）。这些技术组件相互配合，形成了一套完整的自动化固件仿真解决方案。

实践贡献：本研究在多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）上验证了方法的有效性，展示了该方法在实际固件仿真任务中的可行性和实用性。实验结果表明，该方法能够支持多种类型的固件在仿真环境中正常运行，为固件安全测试提供了新的技术途径，对于提升物联网设备的安全性具有实际意义。

5.4 局限性与不足

尽管本研究取得了一定的成果，但仍存在以下局限性和不足：

（1）对固件源码的依赖

当前方法主要针对有源码的固件进行分析和替换。对于闭源固件，由于缺乏源码信息，静态分析模块难以准确识别驱动函数的语义，LLM 生成的替代代码质量也会受到影响。虽然理论上可以对二进制固件进行反编译和分析，但反编译结果的准

确性和可读性限制了方法的效果。如何在没有源码的情况下进行高质量的驱动替换，是未来需要解决的问题。

（2）大语言模型的成本和效率问题

大语言模型调用存在显著的成本和时间开销。对于包含大量驱动函数的固件，进行完整的分析和替换可能需要调用 LLM 数十甚至上百次，这不仅带来经济成本，也影响处理效率。此外，LLM 生成代码的质量存在不确定性，有时需要多次迭代才能获得满意的结果。如何降低对 LLM 的依赖，提高处理效率，是实际应用中需要考虑的问题。

（3）平台和操作系统覆盖范围有限

目前方法支持的平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）仍然有限。物联网设备生态复杂，存在大量其他平台（如 ESP32、Nordic nRF 系列等）和操作系统（如 Zephyr、Mbed OS 等）。不同平台的外设实现和驱动接口存在差异，需要针对性地扩展静态分析规则和提示词模板。扩展方法的覆盖范围，使其能够支持更多的嵌入式平台，是未来工作的重要方向。

（4）复杂外设行为的模拟精度

对于一些复杂的外设（如网络控制器、USB 控制器等），其内部状态机逻辑复杂，状态转换涉及多个寄存器的协调操作和时序约束。当前方法生成的替代代码可能无法完全准确地模拟这些复杂行为，导致固件在某些场景下的运行结果与真实硬件存在偏差。如何提高对复杂外设行为的模拟精度，是提升仿真质量的关键。

5.5 未来工作展望

针对上述局限性，未来的研究可以从以下几个方向展开：

（1）支持无源码固件的驱动替换

研究如何在缺乏源码的情况下进行驱动函数的识别和替换。可以探索基于二进制分析的驱动函数定位技术，利用反编译工具（如 Ghidra、IDA Pro）提取函数的中间表示，结合模式匹配和机器学习方法识别驱动函数。对于 LLM 输入，可以尝试使用反编译后的伪代码或控制流图作为输入，引导模型生成替代代码。此外，还可以

研究基于动态分析的驱动行为学习方法，通过观察固件在真实硬件上的执行轨迹，推断驱动函数的语义和行为模式。

（2）降低大语言模型的调用成本

探索降低 LLM 调用成本的多种策略。一方面，可以通过模型微调（Fine-tuning）技术，在嵌入式驱动代码数据集上训练专门的代码生成模型，使其更适合驱动替换任务，从而减少对通用大模型的依赖。另一方面，可以设计更高效的提示词策略，通过 Few-shot Learning 或检索增强生成（RAG）技术，利用已有的驱动替换案例指导新代码的生成，减少迭代次数。此外，还可以研究代码缓存和复用机制，对于相似功能的驱动函数，直接复用已有的替代代码，避免重复调用 LLM。

（3）扩展平台和操作系统的支持范围

逐步扩展方法对不同嵌入式平台和操作系统的支持。可以采用增量扩展的策略，优先支持市场占有率较高的平台（如 ESP32、nRF52 等）和操作系统（如 Zephyr、Mbed OS 等）。对于新平台的适配，需要收集该平台的固件样本，分析其外设实现特点和驱动接口规范，扩展静态分析规则库和提示词模板库。此外，还可以研究自动化的平台适配技术，通过分析平台的 SDK 和头文件，自动生成适配规则。

（4）提升复杂外设的模拟精度

针对复杂外设的模拟问题，可以研究基于状态机学习的建模方法。通过观察固件与外设的交互序列，自动学习外设的状态机模型，包括状态集合、转移条件和输出行为。对于时序敏感的操作，可以引入时间建模机制，在仿真环境中模拟硬件的时间行为。此外，还可以探索混合仿真策略，对于关键的复杂外设，采用高精度的仿真模型；对于简单的外设，采用 LLM 生成的替代代码，在精度和效率之间取得平衡。

（5）安全性验证与保障

引入形式化验证方法，确保替换代码的安全性和正确性。可以研究基于符号执行的验证技术，对 LLM 生成的替代代码进行自动化的安全性检查，确保其不会引入新的安全漏洞。此外，还可以建立驱动替换代码的测试基准（Test Oracle），通过与真实硬件行为的对比，验证仿真结果的正确性。对于关键应用场景（如工业控制、医疗设备等），需要更严格的验证机制，确保仿真环境的可靠性。

致 谢

本论文的完成离不开许多人的帮助和支持，在此向他们表达我最诚挚的感谢。

首先，我要感谢我的导师。在论文选题、研究方法、实验设计等各个方面，导师都给予了我悉心的指导和宝贵的建议。导师严谨的学术态度、渊博的专业知识和敏锐的学术洞察力，使我受益匪浅。

感谢实验室的各位同学。在研究过程中，我们相互讨论、相互帮助，共同解决了很多难题。特别感谢在实验环境搭建、测试用例设计等方面给予我帮助的同学们。

感谢我的家人。他们一直以来的理解、支持和鼓励是我完成学业的坚强后盾。

最后，感谢所有在论文评审和答辩过程中提出宝贵意见的专家学者，你们的建议使本论文得以完善。

作者

于华中科技大学

攻读硕士学位期间发表的学术论文

1)（待补充）